20 projets d'intelligence artificielle

Catalogue de 20 projets complexes et progressifs (fondements, ML classique, vision, NLP, séries temporelles, recommandation, IA générative, LLM, RAG, agents, parole, RL, MLOps, sécurité et éthique, multimodal, passage à l'échelle, produit) avec objectifs, notions à apprendre, outils, ressources, étapes, documentation et plateforme commune AI Lab Hub.

Comment lire ce catalogue

Ce document applique la même logique que les catalogues « développement logiciel », « réseaux, systèmes et sécurité » et « télécoms », cette fois au domaine de l'intelligence artificielle : 20 projets complexes, classés par difficulté croissante, qui couvrent les fondements mathématiques, l'apprentissage supervisé et non supervisé, la vision, le langage, les séries temporelles, la recommandation, les modèles génératifs, les grands modèles de langage (entraînement, fine-tuning, RAG, agents), la parole, l'apprentissage par renforcement, le MLOps, la sécurité et l'éthique de l'IA, le multimodal et le passage à l'échelle. Pour chaque projet : objectif et exigences, notions théoriques à rechercher avant de pratiquer, outils et plateformes, ressources d'apprentissage, étapes de réalisation, compétences acquises, façon de documenter (de la théorie à la pratique) et manière de le mettre en valeur, soit par une plateforme dédiée quand le projet s'y prête, soit via la plateforme commune décrite plus bas.

Projets

20

4 fondements et ML classique · 6 deep learning par domaine · 5 LLM et IA générative · 5 industrialisation, sécurité et synthèse

Matériel

0 à 20 €/mois

PC 16 Go de RAM ; GPU gratuit sur Google Colab ou Kaggle (30 h/semaine) ; un GPU personnel (RTX 3060/4060, 8-12 Go) accélère nettement les projets 09 à 13 mais n'est pas indispensable

Ordre conseillé. Le projet 01 (environnement d'expérimentation et première baseline) est le socle ; le projet 02 (algorithmes écrits à la main) donne les fondements dont tout le reste dépend. Ensuite : *Fondations* (01 à 06), *Avancé* (07 à 14), *Expert* (15 à 20). Le projet 20 (produit IA complet avec gouvernance) réutilise tous les autres. Les projets 04, 07 et 20 s'appuient sur les données produites par les catalogues réseaux et télécoms si vous les avez réalisés : c'est l'occasion de faire dialoguer vos portfolios.

Règle absolue : l'évaluation honnête. Un modèle qui affiche 99 % de précision est presque toujours un modèle mal évalué (fuite de données, jeu de test contaminé, métrique inadaptée à des classes déséquilibrées). Chaque projet impose une baseline naïve, un protocole de validation écrit *avant* l'entraînement, et des métriques justifiées. C'est la rigueur d'évaluation, pas l'architecture du modèle, qui distingue un praticien sérieux.

Méthodologie commune : de la théorie à la pratique documentée

Un projet d'IA produit du code, mais surtout des données préparées, des expériences comparées, des métriques et un modèle dont il faut expliquer les limites. Chaque projet suit six phases ; chaque phase produit un document versionné dans un dépôt Git (`ia/<numero>-<nom>/`), les expériences sont suivies dans un outil de tracking et chaque modèle publié est accompagné d'une **fiche de modèle** (model card).

Phases communes à tous les projets et livrable de chaque phase

Phase	Activités	Livrable
1. Étude théorique	Lire les articles fondateurs et un cours, refaire les démonstrations clés, écrire les formules avec ses mots, construire un glossaire.	`01-theorie.md` : fiches de concepts, formules, articles lus avec résumé critique
2. Cadrage et données	Formuler le problème (tâche, métrique métier, métrique technique), choisir et explorer les données (EDA), identifier biais et fuites potentielles, définir le protocole de validation, rédiger la fiche de données.	`02-donnees.md`, carnet d'exploration, `datasheet.md`, protocole de validation
3. Baseline et expérimentation	Baseline naïve puis simple, expériences suivies (MLflow / W&B), hypothèses écrites avant chaque essai, ablations.	`03-experiences.md`, tableau d'expériences exporté du tracker
4. Modélisation	Modèle retenu, architecture justifiée, hyperparamètres, code propre et testé, reproductibilité (graine, versions, données figées).	Code du modèle, `configs/`, tests, `Dockerfile`
5. Évaluation et analyse	Métriques sur jeu de test jamais vu, intervalles de confiance, analyse d'erreurs qualitative, robustesse, équité, explicabilité, comparaison à la baseline et à l'état de l'art.	`04-evaluation.md`, `model_card.md`, analyse d'erreurs
6. Déploiement, retour d'expérience et publication	API ou démo interactive, monitoring, coûts d'inférence ; difficultés, erreurs à éviter, limites ; page sur la plateforme, article, démo publique.	`05-deploiement.md`, `06-retex.md`, démo publiée, article

+ Modèle de documentation d'un projet IA (structure de la page et de la fiche de modèle)

Le réflexe qui distingue un profil IA sérieux. Écrire l'hypothèse avant l'expérience, comparer toujours à une baseline, ne toucher au jeu de test qu'une seule fois à la fin, et documenter autant les échecs que les réussites. Un tableau d'expériences honnête avec des essais ratés vaut plus qu'un score isolé sans contexte.

Plateforme commune : « AI Lab Hub », le portfolio vivant des modèles

Beaucoup de projets d'IA produisent un modèle, des métriques et des carnets, pas une application. La plateforme commune les rend visibles, testables et comparables : chaque projet a sa page, son tableau d'expériences, sa fiche de modèle et surtout une **démo interactive** où le visiteur peut essayer le modèle. Elle reprend le principe de LabHub et TelcoLab Hub, adapté au cycle de vie des modèles.

Composants de la plateforme commune AI Lab Hub

Composant	Technologie proposée	Rôle
Site de documentation	MkDocs Material (ou Docusaurus) + MathJax, généré depuis le dépôt `ia/`	Une page par projet, formules rendues, carnets convertis en pages, publication automatique par CI.
Application « portfolio de modèles »	Next.js + FastAPI · PostgreSQL	Catalogue des projets, fiches de modèles, tableau d'expériences importé depuis MLflow, comparaison de modèles, page « CV technique ».
Suivi d'expériences et registre de modèles	MLflow auto-hébergé (ou Weights & Biases en gratuit)	Toutes les expériences de tous les projets, artefacts, versions de modèles, étape de promotion (staging → production).
Démos interactives	Gradio ou Streamlit, hébergées sur Hugging Face Spaces (gratuit) et intégrées en iframe	Chaque modèle est essayable en ligne : classifier une image, poser une question au RAG, générer une image, jouer contre l'agent RL.
Service d'inférence	FastAPI + ONNX Runtime / vLLM, conteneurs, GPU optionnel	API unifiée pour les modèles déployés, avec latence et coût mesurés et affichés.
Monitoring	Prometheus + Grafana, Evidently AI pour la dérive	Montre que les modèles tournent réellement : requêtes, latence, dérive des données, qualité au fil du temps.
Hub de modèles et de données	Compte Hugging Face (modèles, datasets, Spaces)	Publication des modèles fine-tunés et des jeux de données avec leurs fiches — visibilité dans la communauté.
Dépôts publics	GitHub : `ia-labs`, `ml-from-scratch`, `llm-experiments`, `mlops-platform`	Preuves de code, carnets, pipelines.
Blog et vidéos	Articles de retour d'expérience, courtes vidéos de démo	Un article par projet ; la vulgarisation est très valorisée en IA.
Hébergement	VPS pour l'application et MLflow ; GPU à la demande (RunPod, Vast.ai, Colab) pour l'entraînement ; Hugging Face Spaces pour les démos	Coût maîtrisé : les démos lourdes tournent chez Hugging Face, l'application légère sur le VPS.

Fonctionnalités de l'application AI Lab Hub

- Catalogue des 20 projets avec statut, niveau, domaine, données utilisées, résultat principal contre la baseline.
- Fiche projet : résumé, théorie, données, expériences, évaluation, fiche de modèle, démo intégrée, liens.
- **Tableau d'expériences** synchronisé depuis MLflow : chaque essai avec hypothèse, configuration, métriques, durée et coût ; graphique des progrès au fil des essais.
- **Comparateur de modèles** : baseline vs modèles successifs vs état de l'art publié, avec intervalles de confiance.
- Galerie de démos interactives (Spaces intégrés) — la fonctionnalité qui fait la différence pour un visiteur.
- Bibliothèque de fiches de modèles et de fiches de données.
- Tableau de bord de production : modèles déployés, requêtes, latence, dérive détectée.
- Glossaire IA transversal et bibliographie annotée des articles lus.
- Matrice de compétences (maths, données, ML classique, deep learning, NLP/LLM, vision, MLOps, éthique) alimentée par les projets terminés ; page « CV technique » exportable, certifications (Google ML Engineer, AWS ML Specialty, DeepLearning.AI).

Étapes de réalisation de la plateforme

1. Semaine 1 : dépôt `ia/` avec le modèle de documentation et la fiche de modèle type, MkDocs + MathJax, CI vers GitHub Pages ; MLflow installé (dès le projet 01, pour tracer toutes les expériences depuis le début).
2. Semaine 2 : modèle de données (projet, expérience, modèle, jeu de données, démo, compétence), API FastAPI, import automatique depuis MLflow.
3. Semaine 3 : interface catalogue et fiches, intégration des Spaces Hugging Face.
4. Semaine 4 : comparateur de modèles, tableau de bord de production, glossaire.
5. Ensuite : chaque projet terminé publie sa démo, ses expériences et sa fiche ; la plateforme grandit avec vous.

Vue d'ensemble des 20 projets

Les 20 projets, leur domaine, leurs outils principaux, le niveau et le mode de valorisation

N°	Projet	Domaine	Outils principaux	Niveau	Valorisation
01	Environnement d'expérimentation et première baseline tabulaire	Socle	Python, Jupyter, pandas, scikit-learn, MLflow, Docker	Fondations	Plateforme commune (socle)
02	Machine learning écrit à la main : des maths au réseau de neurones	Fondements	NumPy, calcul différentiel, algèbre linéaire, Jupyter	Fondations	Plateforme dédiée possible (visualiseur d'apprentissage)
03	ML supervisé tabulaire avancé : compétition de bout en bout	ML classique	XGBoost/LightGBM/CatBoost, Optuna, SHAP, Kaggle	Fondations	Plateforme commune
04	Apprentissage non supervisé et détection d'anomalies	ML classique	k-means, DBSCAN, UMAP, Isolation Forest, autoencodeurs	Fondations	Plateforme commune (tableau d'anomalies)
05	Vision par ordinateur : classification, détection et déploiement embarqué	Deep learning · vision	PyTorch, torchvision, YOLO, ONNX, Raspberry Pi	Fondations	Plateforme dédiée possible (démo caméra)
06	Traitement du langage : du sac de mots au fine-tuning de BERT	Deep learning · NLP	spaCy, Hugging Face Transformers, embeddings	Fondations	Plateforme commune + démo
07	Séries temporelles et prévision	Deep learning · temps	statsmodels, Prophet, LSTM/TFT, Darts, Nixtla	Avancé	Plateforme commune (prévisions en direct)
08	Système de recommandation	Deep learning · reco	implicit, LightFM, modèle two-tower, FAISS	Avancé	Plateforme dédiée possible (site de recommandation)
09	Modèles génératifs d'images : VAE, GAN et diffusion	IA générative	PyTorch, Diffusers, Stable Diffusion, LoRA	Avancé	Plateforme dédiée possible (galerie générative)
10	Construire un petit GPT : le	LLM	PyTorch, tokenisation BPE, nanoGPT	Avancé	Plateforme commune + démo

N°	Projet	Domaine	Outils principaux	Niveau	Valorisation
	transformer de zéro				
11	Fine-tuning, quantification et inférence locale de LLM	LLM	Hugging Face, PEFT/LoRA, TRL, llama.cpp, vLLM	Avancé	Plateforme commune + modèle publié
12	Assistant documentaire RAG avec évaluation	LLM · applications	LangChain/LlamaIndex, base vectorielle, RAGAS	Avancé	Plateforme dédiée (assistant sur vos propres catalogues)
13	Agents IA, outils et garde-fous	LLM · agents	Function calling, MCP, LangGraph, évaluation d'agents	Avancé	Plateforme dédiée possible (agent d'exploitation)
14	Parole : reconnaissance, synthèse et assistant vocal	Deep learning · audio	Whisper, wav2vec2, Coqui/Piper TTS, VAD	Avancé	Plateforme dédiée possible (assistant vocal)
15	Apprentissage par renforcement	RL	Gymnasium, Stable-Baselines3, DQN, PPO, environnements maison	Expert	Plateforme commune (vidéos d'agents)
16	MLOps : pipeline de bout en bout, déploiement et monitoring	Industrialisation	DVC, MLflow, FastAPI, Kubernetes, Evidently, CI/CD	Expert	Plateforme dédiée (la plateforme MLOps elle-même)
17	Sécurité et IA responsable : attaques, biais, explicabilité, conformité	Sécurité · éthique	ART, Garak, Fairlearn, SHAP, AI Act	Expert	Plateforme commune (rapports d'audit)
18	IA multimodale : vision-langage et compréhension de documents	Multimodal	CLIP, VLM (LLaVA, Qwen-VL), OCR, LayoutLM	Expert	Plateforme dédiée possible (analyse de documents)
19	Passage à l'échelle : entraînement distribué et inférence optimisée	Performance	PyTorch DDP/FSDP, mixed precision, ONNX/TensorRT, vLLM, profiling	Expert	Plateforme commune (rapports de performance)
20	Produit IA complet : de la	Synthèse	Tout le catalogue + gestion des risques et documentation	Expert	Plateforme dédiée (le produit

N°	Projet	Domaine	Outils principaux	Niveau	Valorisation
	donnée à la gouvernance		réglementaire		lui-même)

+ Plateformes d'apprentissage, cours, livres et certifications de référence (utilisés dans plusieurs projets)

01 · Environnement d'expérimentation et première baseline tabulaire

Domaine	Socle : outillage, données, méthodologie d'expérimentation
Niveau	Fondations (socle de tous les projets)
Durée indicative	2 à 3 semaines
Valorisation	Plateforme commune : page « le laboratoire », gabarit de projet réutilisable

Objectif et exigences

Mettre en place l'environnement et la discipline qui serviront aux dix-neuf projets suivants : environnement Python reproductible, GPU accessible (local ou cloud), suivi d'expériences, gestion de versions des données, gabarit de projet, et un premier problème complet traité de bout en bout sur des données tabulaires pour roder toute la chaîne. Exigences : environnement reconstructible depuis Git en une commande, toute expérience automatiquement tracée (paramètres, métriques, artefacts, version du code et des données), un gabarit de projet réutilisable, et un premier modèle complet — avec analyse exploratoire, baseline naïve, modèle simple, évaluation honnête et fiche de modèle.

Notions à rechercher et apprendre

- Le cycle de vie d'un projet ML (cadrage, données, modèle, évaluation, déploiement, monitoring) et la différence entre développement logiciel classique et développement de modèles (le comportement dépend des données, pas seulement du code).
- Types de problèmes (classification, régression, clustering, ranking, génération) et vocabulaire (variables, cibles, exemples, entraînement/validation/test, surapprentissage, généralisation).
- Reproductibilité : graines aléatoires, versions des bibliothèques, versionnement des données (pourquoi Git ne suffit pas pour des fichiers de plusieurs Go), conteneurs, environnements.
- Analyse exploratoire (EDA) : distributions, valeurs manquantes, corrélations, valeurs aberrantes, déséquilibre de classes.
- Découpage et validation : train/validation/test, validation croisée, stratification, **fuite de données** (data leakage) et ses formes sournoises (normaliser avant de découper, variables qui contiennent la cible, doublons entre train et test).
- Métriques de base et leurs pièges : accuracy sur classes déséquilibrées, précision/rappel/F1, ROC-AUC vs PR-AUC, RMSE/MAE, pourquoi il faut une baseline naïve (prédire la classe majoritaire, la moyenne).
- Suivi d'expériences : ce qu'il faut tracer et pourquoi.

Outils, plateformes et ressources

- **Environnement** : Python 3.11+, gestion de dépendances avec uv ou Poetry (ou conda si besoin de CUDA), Jupyter Lab, VS Code ; Docker avec image GPU (nvidia/cuda) ; pre-commit avec ruff et black.
- **Calcul** : Google Colab et Kaggle Notebooks (GPU gratuits), un compte RunPod ou Vast.ai pour du GPU à la demande (quelques centimes par heure), GPU local si disponible.

- **Bibliothèques** : pandas, Polars (rapide, moderne), NumPy, scikit-learn, Matplotlib/Seaborn/Plotly, ydata-profiling pour l'EDA automatique.
- **Suivi et versionnement** : MLflow (auto-hébergé, tracking + registre de modèles) ou Weights & Biases (gratuit pour un usage personnel) ; DVC pour versionner les données et les pipelines ; cookiecutter-data-science pour le gabarit.
- **Données** : un jeu tabulaire réaliste et imparfait (par exemple les données de prêts Home Credit sur Kaggle, ou le jeu « Adult » de l'UCI, ou les données publiques de la SNCF/RTE sur data.gouv.fr) ; évitez Iris et Titanic, trop propres pour apprendre quoi que ce soit.
- **Ressources** : Kaggle Learn (« Intro to Machine Learning », « Intermediate ML », « Data Cleaning » — quelques heures chacun), premiers chapitres de Géron (*Hands-On ML*, chapitres 1 à 3 : la démarche complète sur un projet), documentation MLflow et DVC, « Made With ML » (partie sur la structuration de projets), article « Hidden Technical Debt in Machine Learning Systems » (Sculley et al., Google).

Étapes de réalisation

1. **Théorie (3 à 4 jours)** : fiches sur le cycle de vie, les découpages et la fuite de données, les métriques ; lister vingt façons de fuiter des données et comment les détecter — cette liste vous servira pendant tout le catalogue.
2. **Environnement** : dépôt `ia/`, environnement Python versionné, image Docker GPU, vérification que PyTorch voit le GPU (local ou Colab), pre-commit, tests avec pytest.
3. **Gabarit de projet** : structure standard (`data/`, `notebooks/`, `src/`, `configs/`, `tests/`, `docs/`), configuration par fichiers YAML (Hydra ou simple), script d'entraînement paramétrable, logging MLflow intégré, fiche de modèle type. Tous les projets suivants seront créés depuis ce gabarit.
4. **Suivi d'expériences** : MLflow déployé (sur votre VPS ou en local avec stockage d'artefacts), DVC initialisé avec un dépôt distant (S3 compatible, Google Drive ou disque), première expérience factice tracée de bout en bout.
5. **Premier problème — EDA** : choisir le jeu tabulaire, écrire la fiche de données (origine, licence, taille, variables, biais connus), réaliser une exploration complète (distributions, manquants, corrélations, déséquilibre) et en tirer cinq hypothèses de modélisation écrites.
6. **Protocole de validation** : écrire *avant* tout modèle le protocole (découpage stratifié ou temporel, validation croisée à 5 plis, métrique principale justifiée, métriques secondaires), et figer le jeu de test.
7. **Baselines** : baseline naïve (classe majoritaire / moyenne), puis régression logistique ou linéaire avec prétraitement dans un `Pipeline` scikit-learn (pour éviter la fuite lors de la normalisation), tracées dans MLflow.
8. **Modèle simple** : forêt aléatoire ou gradient boosting avec quelques hyperparamètres, comparaison aux baselines avec intervalles de confiance (bootstrap sur le jeu de test), analyse d'erreurs (sur quels exemples le modèle se trompe et pourquoi), importance des variables.
9. **Validation de la chaîne** : détruire l'environnement, le reconstruire depuis Git, relancer l'entraînement et obtenir exactement les mêmes métriques ; écrire la première fiche de modèle ; publier la page projet.

Compétences acquises

- Mise en place d'un environnement de data science reproductible et outillé (GPU, tracking, versionnement des données).
- Démarche complète sur un problème réel : EDA, protocole, baseline, modèle, évaluation, documentation.
- Détection et prévention des fuites de données — l'erreur la plus fréquente des débutants.
- Choix et interprétation des métriques, intervalles de confiance.
- Discipline d'expérimentation qui servira à tous les projets suivants.

Documentation et mise en valeur

Livrables : gabarit de projet publié (réutilisable par d'autres — c'est déjà une contribution), procédure d'installation, carnet d'EDA, fiche de données, protocole de validation, tableau d'expériences MLflow, première fiche de modèle, liste « vingt fuites de données ». Sur la plateforme commune : la page « Le laboratoire » (environnement, matériel, méthode) que toutes les autres pages référencent, et le gabarit en téléchargement. Un lecteur doit comprendre en deux minutes comment vous travaillez : c'est ce qui rend crédibles tous les résultats publiés ensuite.

02 · Machine learning écrit à la main : des maths au réseau de neurones

Domaine	Fondements mathématiques et algorithmiques
Niveau	Fondations (le plus théorique, le plus rentable)
Durée indicative	6 semaines
Valorisation	Plateforme dédiée possible : visualiseur interactif d'apprentissage

Objectif et exigences

Implémenter soi-même, en NumPy uniquement, les algorithmes fondamentaux du machine learning et un réseau de neurones complet avec rétropropagation, puis vérifier chaque implémentation contre scikit-learn ou PyTorch. Sans ce projet, le deep learning reste une boîte noire et le débogage des projets suivants devient une loterie. Exigences : chaque algorithme écrit sans bibliothèque de ML, gradients vérifiés numériquement, résultats identiques (à la tolérance près) aux implémentations de référence, une mini-bibliothèque d'autodifférenciation fonctionnelle, et des carnets qui expliquent chaque formule avant de la coder.

Notions à rechercher et apprendre

- **Algèbre linéaire** : vecteurs, matrices, produit matriciel, transposée, inverse, normes, valeurs et vecteurs propres, décomposition en valeurs singulières (SVD), projection ; interprétation géométrique.
- **Calcul différentiel** : dérivée, gradient, règle de la chaîne (le cœur de la rétropropagation), jacobienne, hessienne, dérivées matricielles courantes.
- **Probabilités et statistiques** : variables aléatoires, distributions usuelles (Bernoulli, gaussienne, catégorielle), espérance et variance, maximum de vraisemblance (et pourquoi l'entropie croisée et l'erreur quadratique en découlent), théorème de Bayes, biais et variance.
- **Optimisation** : descente de gradient (batch, stochastique, mini-batch), taux d'apprentissage, momentum, RMSProp, Adam, convexité, minima locaux et points-selles, ordonnancement du taux d'apprentissage.
- **Modèles linéaires** : régression linéaire (solution analytique et par gradient), régression logistique, softmax, régularisation L1 et L2 et leur interprétation.
- **Autres algorithmes classiques** : k plus proches voisins, arbres de décision (critères d'impureté, construction récursive), k-means, analyse en composantes principales (via la SVD), naïve Bayes, SVM (notion de marge, noyaux).
- **Réseaux de neurones** : perceptron multicouche, fonctions d'activation (sigmoïde, tanh, ReLU et ses variantes) et leurs dérivées, initialisation (Xavier, He) et pourquoi elle compte, rétropropagation détaillée couche par couche, gradients qui explosent ou disparaissent, normalisation par lots, dropout.
- **Autodifférenciation** : graphe de calcul, mode inverse, comment PyTorch calcule les gradients — à comprendre en écrivant un mini-autograd.
- **Diagnostic** : courbes d'apprentissage, sous- et surapprentissage, compromis biais-variance, vérification de gradient par différences finies.

Outils, plateformes et ressources

- **Outils** : NumPy, Matplotlib, Jupyter ; scikit-learn et PyTorch uniquement pour *vérifier* vos implémentations.

- **Ressources principales** : *Neural Networks: Zero to Hero* d'Andrej Karpathy (YouTube — la série « micrograd » et « makemore » est exactement ce projet, suivez-la en codant vous-même) ; 3Blue1Brown (séries algèbre linéaire, calcul, réseaux de neurones) ; *Mathematics for Machine Learning* (Deisenroth, Faisal, Ong — gratuit) ; cours de Andrew Ng (Machine Learning Specialization) pour la démarche ; *Dive into Deep Learning* (d2l.ai) pour les implémentations commentées.
- **Compléments** : *The Matrix Cookbook* (formulaire de dérivées matricielles), CS231n notes sur la rétropropagation, StatQuest pour les intuitions, exercices de « ML from scratch » sur GitHub pour comparer.

Étapes de réalisation

1. **Semaine 1 — mathématiques** : carnets sur l'algèbre linéaire (implémenter la PCA via la SVD et visualiser les projections), le gradient (visualiser un champ de gradient, descendre une fonction 2D, observer l'effet du taux d'apprentissage), la vraisemblance (dériver l'entropie croisée à partir de la loi de Bernoulli).
2. **Semaine 2 — modèles linéaires** : régression linéaire par solution analytique puis par descente de gradient (comparer), régression logistique avec gradient dérivé à la main, softmax multiclasse, régularisation ; vérifier contre scikit-learn ; tracer les frontières de décision.
3. **Semaine 3 — algorithmes classiques** : k-NN, arbre de décision (construction récursive, critère de Gini), k-means, naïve Bayes ; comparer avec scikit-learn sur les mêmes données ; comprendre les cas où chacun échoue.
4. **Semaine 4 — réseau de neurones à la main** : perceptron multicouche en NumPy avec passe avant et rétropropagation écrites couche par couche, vérification de gradient par différences finies (un écart relatif inférieur à 1e-7 valide votre implémentation), entraînement sur MNIST jusqu'à ~98 % ; expérimenter initialisations et activations et observer les gradients qui disparaissent avec la sigmoïde sur 10 couches.
5. **Semaine 5 — mini-autograd** : écrire une classe `Tensor` qui construit un graphe de calcul et calcule les gradients en mode inverse (comme micrograd), réimplémenter le réseau avec cet autograd, comparer les gradients avec PyTorch sur les mêmes valeurs — ils doivent coïncider.
6. **Semaine 6 — optimiseurs et diagnostic** : implémenter SGD avec momentum, RMSProp et Adam, comparer leurs trajectoires sur des fonctions test et sur MNIST ; produire les courbes d'apprentissage typiques du sous- et surapprentissage et un guide de diagnostic « quelle courbe indique quel problème ».
7. **Validation** : tableau « algorithme / votre implémentation / référence / écart » pour tout le projet, et une réponse écrite à la question « expliquez la rétropropagation à un développeur qui n'a jamais fait de ML » — le vrai test de compréhension.

Compétences acquises

- Compréhension démontrable des mathématiques du machine learning, formules et intuitions.
- Maîtrise de la rétropropagation et de l'autodifférenciation — vous saurez déboguer un entraînement qui diverge.
- Connaissance des algorithmes classiques par leur implémentation.

- Réflexes de diagnostic (courbes, gradients, vérification numérique).
- Une base solide pour lire les articles de recherche des projets suivants.

Documentation et mise en valeur

Livrables : bibliothèque `ml-from-scratch` publiée et testée, carnets pédagogiques (formule → intuition → code → vérification), mini-autograd, guide de diagnostic des courbes d'apprentissage, article « ce que j'ai compris en écrivant la rétropropagation à la main ». Plateforme dédiée envisageable : un **visualiseur interactif d'apprentissage** dans le navigateur (Pyodide ou JavaScript) où le visiteur règle taux d'apprentissage, architecture et optimiseur, et regarde la frontière de décision et la fonction de perte évoluer en direct — à la manière du « TensorFlow Playground », mais construit par vous. Très pédagogique, statique donc gratuit à héberger, et une preuve de compréhension profonde.

03 · ML supervisé tabulaire avancé : compétition de bout en bout

Domaine	Machine learning classique, données structurées
Niveau	Fondations (le plus utilisé en entreprise)
Durée indicative	4 à 5 semaines
Valorisation	Plateforme commune : rapport de compétition, classement obtenu, analyse d'erreurs

Objectif et exigences

Traiter un problème tabulaire difficile comme dans une compétition Kaggle et en entreprise : ingénierie des variables, gradient boosting, optimisation des hyperparamètres, validation rigoureuse, ensembles, interprétabilité, et soumission publique pour se confronter à un classement réel. Les données tabulaires représentent l'essentiel des cas d'usage en entreprise et le gradient boosting y reste l'état de l'art : ce projet forme au métier tel qu'il se pratique. Exigences : participation à une compétition active ou récente, protocole de validation locale corrélé au classement public, au moins trois familles de modèles comparées, un ensemble final, interprétation avec SHAP, et un rapport reproductible de bout en bout.

Notions à rechercher et apprendre

- **Ingénierie des variables** : encodage des catégories (one-hot, ordinal, target encoding et son risque de fuite, encodage par fréquence), traitement des valeurs manquantes (et l'information qu'elles portent), transformations (log, binning), variables d'agrégation et de jointure, variables temporelles (retards, fenêtres glissantes), interactions ; sélection de variables.
- **Gradient boosting** : principe (ajouter des arbres qui corrigent les résidus), XGBoost, LightGBM, CatBoost et leurs différences (croissance par feuille, gestion native des catégories, ordered boosting), hyperparamètres clés (profondeur, taux d'apprentissage, nombre d'arbres, sous-échantillonnage, régularisation) et leurs effets, arrêt précoce.
- **Validation** : validation croisée stratifiée, par groupes (quand plusieurs lignes partagent une entité), temporelle ; décalage entre train et test (adversarial validation pour le détecter) ; pourquoi le classement public n'est pas fiable et comment construire une validation locale fiable.
- **Optimisation des hyperparamètres** : recherche aléatoire vs bayésienne (Optuna, TPE), élagage des essais, budget de calcul, risque de surapprentissage de la validation.
- **Ensembles** : moyenne, moyenne pondérée, stacking et blending, diversité des modèles, pourquoi les ensembles gagnent les compétitions et ce que cela coûte en production.
- **Déséquilibre et métriques** : rééchantillonnage, pondération des classes, seuil de décision, courbes précision-rappel, calibration des probabilités (courbe de fiabilité, Platt, isotonique) — essentielle quand la sortie est une probabilité utilisée pour décider.
- **Interprétabilité** : importance par permutation, valeurs de Shapley (SHAP) globales et locales, dépendance partielle, et leurs limites.
- **Réseaux de neurones sur tabulaire** : pourquoi ils perdent généralement contre le boosting (et les exceptions : TabNet, FT-Transformer, très grandes données).

Outils, plateformes et ressources

- **Bibliothèques** : scikit-learn, XGBoost, LightGBM, CatBoost, Optuna, SHAP, category_encoders, Polars pour les grandes tables, imbalanced-learn.
- **Plateforme** : Kaggle (choisir une compétition tabulaire active — les « Playground Series » mensuelles sont parfaites pour débuter, puis une compétition à enjeu réel) ; alternatives francophones : challenges data ENS, Zindi (Afrique).
- **Suivi** : MLflow du projet 01 pour toutes les expériences.
- **Ressources** : Kaggle Learn (« Feature Engineering », « Intermediate ML »), notebooks des gagnants de compétitions passées (lire au moins cinq solutions gagnantes est un cours en soi), *Approaching (Almost) Any Machine Learning Problem* (Abhishek Thakur, gratuit), documentation LightGBM et CatBoost (les guides de paramètres), article « Why do tree-based models still outperform deep learning on tabular data? » (Grinsztajn et al., 2022), livre *Interpretable Machine Learning* (Molnar, gratuit) pour SHAP.

Étapes de réalisation

1. **Théorie** : fiches sur le gradient boosting (écrire l'algorithme en pseudo-code), la validation et ses pièges, le target encoding et ses fuites, la calibration ; lire cinq solutions gagnantes et en extraire les motifs récurrents.
2. **Cadrage** : choisir la compétition, lire la description et la métrique attentivement (elle dicte tout), écrire la fiche de données, réaliser l'EDA, formuler dix hypothèses de variables.
3. **Validation locale** : construire le schéma de validation adapté (groupes, temps), faire une soumission de baseline et vérifier que le score local et le score public sont corrélés — sinon corriger le schéma avant d'aller plus loin ; réaliser une validation adversariale pour détecter un décalage train/test.
4. **Baselines** : naïve, régression logistique/linéaire, puis LightGBM avec paramètres par défaut : c'est la référence à battre.
5. **Ingénierie des variables** : itérer hypothèse par hypothèse, chaque variable ajoutée est une expérience tracée avec son effet sur la validation ; conserver seulement ce qui apporte ; documenter les variables qui n'ont rien donné (aussi instructif).
6. **Modèles et hyperparamètres** : XGBoost, LightGBM, CatBoost optimisés avec Optuna (budget fixé, par exemple 100 essais), plus un modèle d'une autre famille (régression régularisée ou petit réseau) pour la diversité ; comparer.
7. **Ensemble** : moyenne pondérée puis stacking avec un méta-modèle simple ; mesurer le gain réel par rapport au meilleur modèle seul ; discuter le rapport gain / complexité.
8. **Calibration et seuil** : si la métrique dépend d'un seuil ou de probabilités, calibrer et optimiser le seuil sur la validation uniquement.
9. **Interprétation et analyse d'erreurs** : SHAP global (quelles variables comptent) et local (expliquer dix prédictions individuelles), analyse des pires erreurs, segments où le modèle est faible ; vérifier que les variables les plus importantes sont plausibles (une importance suspecte révèle souvent une fuite).
10. **Soumission et rapport** : soumission finale, classement obtenu documenté honnêtement, rapport reproductible (un script qui régénère la soumission), fiche de modèle.

Compétences acquises

- Maîtrise du gradient boosting, l'outil le plus utilisé sur les données d'entreprise.
- Ingénierie des variables méthodique et validation rigoureuse dans des conditions réalistes.
- Optimisation d'hyperparamètres et construction d'ensembles.
- Calibration et interprétabilité — indispensables pour un modèle utilisé dans des décisions.
- Confrontation à un classement public : une preuve objective et externe de niveau.

Documentation et mise en valeur

Livrables : dépôt reproductible (un script → la soumission), tableau d'expériences complet avec l'effet de chaque variable, comparatif de modèles, analyse SHAP, analyse d'erreurs, classement obtenu, fiche de modèle. Sur la plateforme commune : page projet avec le rapport, le classement (même modeste : l'honnêteté compte plus que le rang) et la démarche ; un article « ce que 100 expériences m'ont appris sur ce jeu de données » a beaucoup plus de valeur qu'un score isolé. Un profil Kaggle actif est un élément de portfolio reconnu par les recruteurs en data science.

04 · Apprentissage non supervisé et détection d'anomalies

Domaine	Machine learning classique, données sans étiquettes
Niveau	Fondations
Durée indicative	4 semaines
Valorisation	Plateforme commune : tableau d'anomalies en direct sur vos propres données de laboratoire

Objectif et exigences

Extraire de la structure de données non étiquetées — segmentation, réduction de dimension, visualisation — puis construire un système de détection d'anomalies appliqué à un flux réel : idéalement les journaux, métriques ou trafic réseau de votre propre laboratoire (catalogues réseaux/télécoms), sinon un jeu public. Le non supervisé est le cas le plus fréquent en pratique (les étiquettes coûtent cher) et la détection d'anomalies est un besoin universel (fraude, pannes, intrusions). Exigences : au moins trois méthodes de clustering et deux de réduction de dimension comparées avec des critères objectifs, un détecteur d'anomalies évalué sur des anomalies connues ou injectées, gestion du flux (détection en quasi temps réel), et une analyse honnête du taux de fausses alertes — le vrai problème de la détection d'anomalies.

Notions à rechercher et apprendre

- **Clustering** : k-means (et le choix de k : coude, silhouette, gap statistic), clustering hiérarchique et dendrogrammes, DBSCAN et HDBSCAN (densité, bruit, pas besoin de k), mélanges gaussiens (GMM, EM), clustering spectral ; métriques internes (silhouette, Davies-Bouldin) et externes (ARI, NMI quand des étiquettes existent) ; pourquoi le clustering est fondamentalement subjectif.
- **Réduction de dimension** : PCA (variance expliquée, interprétation), t-SNE et UMAP (visualisation, et leurs pièges : les distances entre groupes ne signifient rien en t-SNE), autoencodeurs comme réduction non linéaire ; malédiction de la dimension.
- **Détection d'anomalies** : types d'anomalies (ponctuelles, contextuelles, collectives), méthodes statistiques (z-score, IQR, robust covariance), par densité (LOF), par isolation (Isolation Forest), par frontière (One-Class SVM), par reconstruction (autoencodeurs), par prévision (résidus d'un modèle de séries temporelles) ; détection sur flux et fenêtres glissantes ; dérive du concept.
- **Évaluation sans étiquettes** : le problème central ; stratégies : anomalies injectées, étiquetage partiel a posteriori, précision sur les k premières alertes, courbes précision-rappel à faible prévalence ; coût des faux positifs (fatigue des alertes) vs faux négatifs.
- **Prétraitement** : normalisation (indispensable pour les distances), traitement des catégories, agrégation de journaux en variables (par fenêtre de temps, par entité), TF-IDF et embeddings pour des messages de journaux textuels.
- **Passage à l'action** : seuils, priorisation, explication d'une anomalie (quelles variables contribuent), boucle de retour humain.

Outils, plateformes et ressources

- **Bibliothèques** : scikit-learn (clustering, PCA, Isolation Forest, LOF), HDBSCAN, UMAP-learn, PyOD (bibliothèque de référence pour la détection d'anomalies, plus

de 40 algorithmes), PyTorch pour les autoencodeurs, River pour le ML sur flux, Drain3 pour parser des journaux en gabarits.
- **Données** : vos propres journaux (syslog, Suricata, Zeek du catalogue sécurité), métriques Prometheus ou CDR du catalogue télécoms ; sinon jeux publics : CIC-IDS2017 ou UNSW-NB15 (intrusions réseau), NAB (Numenta Anomaly Benchmark, séries temporelles), KDD Cup 99 (historique), jeux de fraude par carte (Kaggle), Loghub (journaux systèmes étiquetés).
- **Visualisation** : Plotly pour les projections interactives, Grafana pour le flux d'anomalies.
- **Ressources** : Géron chapitre 9 (non supervisé), documentation PyOD et son article de synthèse, article « Isolation Forest » (Liu et al., 2008 — court et lisible), « How to Use t-SNE Effectively » (Distill.pub — indispensable pour ne pas mal interpréter), documentation UMAP, cours StatQuest sur PCA et clustering, survey « Deep Learning for Anomaly Detection » (Pang et al., 2021).

Étapes de réalisation

1. **Théorie** : fiches clustering (implémenter k-means et EM en s'appuyant sur le projet 02), réduction de dimension, méthodes d'anomalies ; lire l'article Distill sur t-SNE et refaire ses expériences.
2. **Segmentation — partie 1** : sur un jeu tabulaire (clients, ou les hôtes de votre réseau caractérisés par leur trafic), normaliser, appliquer k-means, hiérarchique, HDBSCAN et GMM, choisir k par plusieurs critères, comparer les partitions (ARI entre méthodes), visualiser en PCA et UMAP, et surtout **interpréter chaque segment** en langage clair (profil type) — l'étape qui donne de la valeur métier.
3. **Réduction de dimension** : comparer PCA, t-SNE, UMAP et un autoencodeur sur les mêmes données (par exemple MNIST puis vos données) ; documenter ce que chaque méthode préserve et déforme.
4. **Détection d'anomalies — préparation** : choisir le flux (journaux ou métriques de votre lab), construire les variables par fenêtre de temps et par entité (nombre d'événements, diversité des destinations, octets, taux d'erreurs…), parser les journaux textuels en gabarits avec Drain3.
5. **Détecteurs** : baseline statistique (z-score robuste), puis Isolation Forest, LOF, One-Class SVM, autoencodeur ; entraîner sur une période « normale », scorer une période contenant des anomalies connues (attaques rejouées depuis le catalogue sécurité, pannes simulées, ou anomalies injectées).
6. **Évaluation** : précision-rappel à faible prévalence, précision sur les 50 premières alertes, délai de détection ; tableau comparatif ; analyse des faux positifs (que sont-ils ? souvent des comportements légitimes rares, à documenter) ; choix du seuil selon un budget d'alertes par jour.
7. **Explication** : pour chaque alerte, indiquer les variables qui contribuent le plus (écart à la normale par variable, SHAP sur Isolation Forest) — une alerte inexplicable est une alerte ignorée.
8. **Flux** : passer en détection continue (fenêtres glissantes, réentraînement périodique, River pour l'incrémental), publier les alertes dans Grafana ou la plateforme, mesurer le taux de fausses alertes sur deux semaines réelles.

Compétences acquises

- Segmentation et interprétation de données non étiquetées, réduction de dimension et visualisation sans se tromper.
- Conception et évaluation d'un système de détection d'anomalies dans des conditions réalistes (sans étiquettes, à faible prévalence).
- Transformation de journaux et métriques bruts en variables exploitables.
- Gestion du compromis alertes / faux positifs et explication des alertes.
- Pont concret entre IA et exploitation/sécurité — un profil très demandé (AIOps, détection de fraude, sécurité).

Documentation et mise en valeur

Livrables : carnets de segmentation avec profils interprétés, comparatif de réduction de dimension, pipeline de détection d'anomalies, tableau d'évaluation, analyse des faux positifs, mesure du taux d'alertes sur deux semaines. Sur la plateforme commune : un **tableau d'anomalies en direct** sur vos propres données de laboratoire avec l'explication de chaque alerte — la preuve que le système fonctionne sur du réel, en continu, et non seulement sur un jeu de données figé. Si vous avez réalisé les catalogues précédents, ce projet relie explicitement vos trois portfolios.

05 · Vision par ordinateur : classification, détection et déploiement embarqué

Domaine	Deep learning · vision
Niveau	Fondations (premier vrai projet deep learning)
Durée indicative	5 à 6 semaines
Valorisation	Plateforme dédiée possible : démo caméra en direct et modèle embarqué

Objectif et exigences

Entraîner des réseaux convolutifs pour la classification puis la détection d'objets sur un jeu de données que vous constituez et annotez vous-même, et déployer le modèle final sur un appareil contraint (Raspberry Pi, téléphone ou navigateur) avec une caméra en temps réel. Exigences : un CNN écrit et entraîné de zéro puis comparé au transfer learning, un jeu de données personnel annoté (au moins 500 images), un détecteur d'objets fonctionnel, un modèle exporté et optimisé tournant à plus de 10 images par seconde sur l'appareil cible, et une évaluation incluant la robustesse (éclairage, angle, objets hors distribution).

Notions à rechercher et apprendre

- **Convolution** : filtres, champs récepteurs, padding, stride, pooling, pourquoi la convolution convient aux images (invariance par translation, partage de paramètres), visualisation des filtres et des activations.
- **Architectures** : LeNet, AlexNet, VGG, ResNet (connexions résiduelles et pourquoi elles permettent la profondeur), Inception, MobileNet et EfficientNet (efficacité), ConvNeXt, Vision Transformer (ViT) et le débat CNN vs transformer.
- **Entraînement** : augmentation de données (recadrage, retournement, couleur, Mixup, CutMix) et pourquoi c'est le levier le plus rentable, normalisation par lots, ordonnancement du taux d'apprentissage (one-cycle, cosinus), régularisation, arrêt précoce, précision mixte.
- **Transfer learning** : pré-entraînement sur ImageNet, gel et dégel des couches, fine-tuning avec taux d'apprentissage différenciés, quand ça marche et quand ça ne marche pas (domaine très différent).
- **Détection d'objets** : boîtes englobantes, IoU, ancres, détecteurs à deux étages (Faster R-CNN) vs une étape (YOLO, SSD, RetinaNet, DETR), non-maximum suppression, mAP et son calcul, focal loss ; segmentation (U-Net, Mask R-CNN, SAM) en notions.
- **Données** : constitution d'un jeu de données, annotation (formats COCO, YOLO, Pascal VOC), qualité et cohérence des annotations, biais de collecte, découpage sans fuite (mêmes objets dans train et test).
- **Déploiement embarqué** : export ONNX, quantification (FP16, INT8), élagage, distillation, TensorFlow Lite, ONNX Runtime, OpenVINO, TensorRT, compromis précision / latence / taille ; inférence dans le navigateur (ONNX Runtime Web, TensorFlow.js).
- **Interprétabilité et robustesse** : Grad-CAM, exemples adverses (notions, approfondies au projet 17), performances par sous-groupe et conditions.

Outils, plateformes et ressources

- **Bibliothèques** : PyTorch et torchvision, timm (centaines de modèles pré-entraînés), Albumentations (augmentation), Ultralytics YOLO (détection, très accessible) ou torchvision detection, Lightning pour structurer l'entraînement, OpenCV.
- **Annotation** : Label Studio (libre, auto-hébergeable), CVAT, Roboflow (gratuit pour de petits jeux, pratique).
- **Matériel** : GPU Colab/Kaggle pour l'entraînement ; Raspberry Pi 4/5 avec caméra (60-100 €), ou un téléphone Android, ou simplement le navigateur ; éventuellement un accélérateur (Coral USB, Hailo) pour aller plus loin.
- **Données** : CIFAR-10 puis un sous-ensemble d'ImageNet (Imagenette de fast.ai) pour apprendre ; ensuite **votre propre jeu** (objets de votre quotidien, équipements de votre lab, plantes, pièces mécaniques — quelque chose qui a du sens pour vous).
- **Ressources** : cours Stanford CS231n (notes et vidéos — la référence), fast.ai partie 1 (chapitres vision, très pratique), *Dive into Deep Learning* chapitres CNN, articles ResNet (He et al., 2015) et YOLO (Redmon et al.), documentation Ultralytics et ONNX Runtime, cours Hugging Face Computer Vision.

Étapes de réalisation

1. **Théorie** : fiches convolution (calculer à la main la taille de sortie et le nombre de paramètres d'une couche), architectures, détection et mAP ; implémenter une convolution 2D en NumPy (lien avec le projet 02) et la vérifier contre PyTorch.
2. **CNN de zéro** : sur CIFAR-10, écrire un petit CNN, l'entraîner, observer le surapprentissage, ajouter augmentation, normalisation par lots et ordonnancement, atteindre 90 %+ ; implémenter un bloc résiduel et mesurer son effet ; visualiser les filtres et Grad-CAM.
3. **Transfer learning** : sur Imagenette puis vos données, fine-tuner un ResNet-50 et un EfficientNet/ConvNeXt via timm, comparer avec l'entraînement de zéro (précision et temps), expérimenter gel/dégel et taux différenciés.
4. **Jeu de données personnel** : définir 5 à 10 classes d'objets, photographier 500+ images en variant conditions et fonds, annoter les boîtes dans Label Studio, écrire la fiche de données (biais de collecte compris), découper par session de prise de vue pour éviter la fuite.
5. **Détection** : entraîner un YOLO (Ultralytics) sur vos données, analyser le mAP par classe, les confusions et les erreurs de localisation ; comparer avec Faster R-CNN de torchvision ; améliorer par augmentation et par correction des annotations douteuses (l'analyse d'erreurs révèle souvent des erreurs d'annotation).
6. **Robustesse** : constituer un jeu de test « difficile » (faible lumière, angles inhabituels, objets partiellement cachés, objets jamais vus) et mesurer la chute de performance ; documenter les limites dans la fiche de modèle.
7. **Optimisation et export** : exporter en ONNX, quantifier en INT8, mesurer précision / latence / taille pour chaque variante (tableau), distiller éventuellement vers un modèle plus petit.
8. **Déploiement** : faire tourner le détecteur en temps réel sur Raspberry Pi avec caméra (ONNX Runtime ou TFLite) ou dans le navigateur (ONNX Runtime Web), mesurer les images par seconde, la température et la consommation ; filmer la démo.

9. **Validation** : tableau complet précision/latence par plateforme, fiche de modèle, vidéo.

Compétences acquises

- Maîtrise des CNN, de l'entraînement deep learning et du transfer learning.
- Constitution et annotation d'un jeu de données — la partie la plus formatrice et la plus proche du réel.
- Détection d'objets et évaluation par mAP, analyse d'erreurs.
- Optimisation et déploiement de modèles sur matériel contraint (ONNX, quantification).
- Évaluation de robustesse et documentation des limites.

Documentation et mise en valeur

Livrables : code d'entraînement, jeu de données publié sur Hugging Face avec sa fiche (si les images le permettent), modèles exportés, tableau précision/latence/taille, Grad-CAM, analyse de robustesse, vidéo du détecteur en temps réel sur Raspberry Pi. Plateforme dédiée envisageable : une **démo caméra dans le navigateur** (ONNX Runtime Web) où le visiteur active sa webcam et voit votre détecteur fonctionner en direct sur sa propre machine — sans serveur, gratuit, spectaculaire. Une vidéo du modèle tournant sur un Raspberry Pi complète parfaitement la page.

06 · Traitement du langage : du sac de mots au fine-tuning de BERT

Domaine	Deep learning · traitement automatique du langage (NLP)
Niveau	Fondations
Durée indicative	5 semaines
Valorisation	Plateforme commune + démo interactive de classification et d'extraction

Objectif et exigences

Parcourir l'évolution complète du NLP sur un problème concret en français : classification de textes (par exemple avis, tickets de support, ou catégorisation d'articles) et extraction d'entités nommées, depuis les approches classiques (sac de mots, TF-IDF) jusqu'au fine-tuning d'un modèle transformer pré-entraîné, en mesurant à chaque étape ce que l'on gagne et ce que l'on paie. Exigences : un pipeline classique solide comme baseline, embeddings de mots entraînés et explorés, fine-tuning d'un modèle BERT francophone (CamemBERT) pour la classification et la NER, comparaison rigoureuse coût/performance, et une démo interactive publiée.

Notions à rechercher et apprendre

- **Prétraitement** : tokenisation (mots, sous-mots — BPE, WordPiece, SentencePiece — et pourquoi les sous-mots ont gagné), normalisation, lemmatisation, mots vides, spécificités du français (accents, élisions, accords).
- **Représentations classiques** : sac de mots, TF-IDF, n-grammes, hachage de variables ; modèles linéaires et naïve Bayes sur ces représentations (baselines redoutablement efficaces).
- **Embeddings de mots** : hypothèse distributionnelle, word2vec (skip-gram, CBOW, échantillonnage négatif), GloVe, fastText (sous-mots, utile pour le français) ; arithmétique des vecteurs, analogies, biais présents dans les embeddings ; limites (un vecteur par mot quel que soit le contexte).
- **Modèles séquentiels** : RNN, LSTM, GRU, problèmes de gradient, attention (Bahdanau) — à comprendre comme l'étape historique qui mène au transformer.
- **Transformers et BERT** : auto-attention, encodeur, pré-entraînement par masquage (MLM), fine-tuning, embeddings contextuels, tokens spéciaux ([CLS], [SEP]), modèles francophones (CamemBERT, FlauBERT) et multilingues (XLM-R, mDeBERTa), distillation (DistilBERT) ; le détail architectural du transformer est approfondi au projet 10.
- **Tâches** : classification (mono et multi-étiquettes), reconnaissance d'entités nommées (schéma BIO, alignement sous-mots/étiquettes), similarité sémantique et embeddings de phrases (Sentence-BERT) — base du RAG du projet 12.
- **Évaluation** : F1 macro/micro, F1 par entité (seqeval), matrice de confusion, analyse d'erreurs linguistique (ironie, négation, domaine), tests de comportement (CheckList).
- **Données textuelles** : collecte, anonymisation, déséquilibre, annotation de NER et accord inter-annotateurs.

Outils, plateformes et ressources

- **Bibliothèques** : scikit-learn (TF-IDF, modèles linéaires), spaCy (pipeline français, NER, lemmatisation), Gensim (word2vec, fastText), Hugging Face Transformers,

Datasets, Tokenizers, Evaluate, seqeval, Sentence-Transformers.
- **Données francophones** : Allociné (avis de films, sentiment), FLUE (benchmark français), WikiNER français ou MultiNERD (NER), tickets ou courriels de votre entreprise anonymisés si possible, Hugging Face Hub (filtre langue française), corpus data.gouv.fr.
- **Annotation** : Label Studio ou Doccano pour annoter des entités.
- **Ressources** : cours Hugging Face NLP (gratuit, exactement ce projet, chapitres 1 à 7), Stanford CS224n (vidéos), *Speech and Language Processing* (Jurafsky, Martin — gratuit en ligne, la référence), *The Illustrated Transformer* et *The Illustrated BERT* (Jay Alammar), article BERT (Devlin et al., 2018), article CamemBERT (Martin et al., 2019), documentation spaCy.

Étapes de réalisation

1. **Théorie** : fiches tokenisation (entraîner un BPE sur un corpus français et observer les sous-mots), TF-IDF, word2vec (implémenter skip-gram avec échantillonnage négatif en PyTorch — lien avec le projet 02), BERT et MLM.
2. **Données** : choisir un problème de classification en français (Allociné pour commencer, puis un corpus qui vous concerne), EDA textuelle (longueurs, vocabulaire, déséquilibre, doublons — très fréquents dans les corpus textuels et source de fuite), protocole de validation, fiche de données.
3. **Baselines classiques** : TF-IDF + régression logistique et naïve Bayes, avec n-grammes ; c'est souvent plus dur à battre qu'on ne croit — noter le score, le temps d'entraînement (secondes) et d'inférence.
4. **Embeddings** : entraîner word2vec et fastText sur votre corpus, explorer (voisins, analogies, biais de genre observés), les utiliser pour une classification par moyenne d'embeddings puis avec un petit LSTM ; comparer aux baselines.
5. **Fine-tuning de CamemBERT — classification** : avec Transformers et le Trainer (ou une boucle PyTorch), fine-tuner CamemBERT-base, optimiser taux d'apprentissage et nombre d'époques, comparer avec DistilCamemBERT (compromis taille/qualité) et avec un modèle multilingue ; tableau performance / temps / coût GPU / latence d'inférence contre toutes les étapes précédentes.
6. **NER** : annoter (ou réutiliser un corpus annoté) des entités pertinentes, gérer l'alignement sous-mots/étiquettes, fine-tuner pour la NER, évaluer avec seqeval par type d'entité, comparer avec spaCy ; analyser les erreurs (entités imbriquées, ambiguïtés).
7. **Embeddings de phrases** : utiliser Sentence-Transformers pour la recherche sémantique sur votre corpus (préparation du projet 12), mesurer la qualité de la recherche sur quelques requêtes annotées.
8. **Analyse d'erreurs et robustesse** : lire cent erreurs et les catégoriser (négation, ironie, hors domaine, étiquette douteuse), tests de comportement (invariance à la casse, aux fautes de frappe, aux noms propres) ; documenter les biais observés.
9. **Démo** : Space Gradio avec classification et surlignage des entités en direct, modèle publié sur Hugging Face avec sa fiche.

Compétences acquises

- Maîtrise de la chaîne NLP complète, des méthodes classiques aux transformers.

- Fine-tuning de modèles pré-entraînés avec l'écosystème Hugging Face — la compétence NLP la plus demandée.
- Compréhension des embeddings et des représentations contextuelles (socle du RAG et des LLM).
- Évaluation et analyse d'erreurs linguistiques, conscience des biais.
- Jugement coût/performance : savoir quand un TF-IDF suffit et quand un transformer se justifie.

Documentation et mise en valeur

Livrables : carnets de chaque étape, tableau comparatif « méthode / F1 / temps d'entraînement / latence / coût » (le livrable clé : il montre le jugement), exploration des embeddings et des biais, modèles publiés sur Hugging Face avec fiches, analyse d'erreurs, démo Gradio. Sur la plateforme commune : la démo intégrée et le tableau comparatif ; un article « faut-il vraiment un transformer pour classer des textes ? » appuyé sur vos mesures fait un excellent contenu de vulgarisation.

07 · Séries temporelles et prévision

Domaine	Deep learning et statistiques · données temporelles
Niveau	Avancé
Durée indicative	4 à 5 semaines
Valorisation	Plateforme commune : prévisions publiées chaque jour et comparées au réel

Objectif et exigences

Prévoir des séries temporelles réelles (consommation électrique, trafic réseau de votre laboratoire, fréquentation, prix) avec des méthodes statistiques puis du deep learning, en respectant la contrainte propre au temps : on n'a jamais accès au futur. Exigences : validation temporelle stricte (backtesting par fenêtres glissantes), baselines naïves saisonnières que tout modèle doit battre, comparaison de méthodes statistiques, de boosting sur variables retardées et de modèles neuronaux, prévision probabiliste (intervalles), prévision multi-séries, et un système qui publie chaque jour une prévision comparée ensuite au réel.

Notions à rechercher et apprendre

- **Structure d'une série** : tendance, saisonnalité (multiple : journalière, hebdomadaire, annuelle), cycles, bruit, stationnarité et tests (ADF, KPSS), différenciation, décomposition (STL), autocorrélation (ACF, PACF).
- **Modèles statistiques** : lissage exponentiel (ETS, Holt-Winters), ARIMA et SARIMA (identification par ACF/PACF, critères AIC/BIC), modèles avec variables exogènes (SARIMAX), Prophet (composants additifs, jours fériés), Theta.
- **Apprentissage automatique** : transformation en problème supervisé par variables retardées (lags), fenêtres glissantes, variables calendaires, gradient boosting pour la prévision (souvent gagnant sur des ensembles de séries, cf. compétition M5), stratégies multi-horizon (direct, récursive).
- **Deep learning** : RNN/LSTM/GRU pour les séquences, réseaux convolutifs temporels (TCN), N-BEATS, Temporal Fusion Transformer, modèles de fondation pour séries (TimesFM, Chronos, Moirai) ; quand ils apportent quelque chose (beaucoup de séries, longues, riches en covariables) et quand ils déçoivent.
- **Prévision probabiliste** : intervalles de prédiction, quantiles, pinball loss, prévision conforme, évaluation par CRPS et couverture — une prévision sans incertitude est inutilisable pour décider.
- **Évaluation** : MAE, RMSE, MAPE et ses pièges (zéros), sMAPE, MASE (relatif à la baseline naïve — la meilleure métrique), backtesting par validation croisée temporelle avec origine glissante, horizon, fuite temporelle (variables calculées avec du futur).
- **Sujets connexes** : détection de ruptures, imputation, données à fréquences irrégulières, hiérarchies de séries (réconciliation), effets des événements exceptionnels.

Outils, plateformes et ressources

- **Bibliothèques** : statsmodels (ARIMA, ETS, décomposition), Prophet, Nixtla (statsforecast, mlforecast, neuralforecast — rapides et modernes), Darts (interface

unifiée), sktime, PyTorch Forecasting, LightGBM, MAPIE pour la prévision conforme.
- **Données** : données publiques françaises (RTE éCO2mix pour la consommation électrique nationale — excellente série multi-saisonnière, data.gouv.fr, Météo-France), vos propres métriques Prometheus du laboratoire (trafic, CPU), jeux de compétition M4/M5 (Kaggle), Monash Forecasting Repository.
- **Ressources** : *Forecasting: Principles and Practice* (Hyndman, Athanasopoulos — gratuit en ligne, la référence absolue, avec code), cours Kaggle « Time Series », article N-BEATS et TFT, blog Nixtla, articles de synthèse de la compétition M5 (ce qui a marché), Rob Hyndman sur les métriques (MASE).

Étapes de réalisation

1. **Théorie** : lire Hyndman chapitres 1 à 9, fiches stationnarité, ARIMA, ETS, backtesting ; implémenter un lissage exponentiel et une autocorrélation à la main.
2. **Données** : choisir deux jeux — une série riche (consommation électrique RTE, pas horaire, plusieurs années) et un ensemble de séries multiples (vos métriques réseau par hôte, ou M5) ; EDA temporelle (décomposition STL, saisonnalités, jours fériés, valeurs manquantes, ruptures) ; fiche de données.
3. **Protocole** : définir l'horizon (par exemple 24 h et 7 jours), la métrique principale (MASE) et le backtesting (origine glissante, 12 fenêtres) ; écrire les baselines naïves (dernière valeur, saisonnière naïve, moyenne mobile).
4. **Modèles statistiques** : ETS, SARIMA (identifié par ACF/PACF puis auto-ARIMA), Prophet avec jours fériés ; résidus analysés (blancs ?) ; backtesting complet.
5. **Boosting** : construire les variables retardées, glissantes et calendaires (en vérifiant l'absence de fuite : chaque variable ne doit utiliser que le passé à la date de prévision), LightGBM multi-horizon direct et récursif, importance des variables.
6. **Deep learning** : LSTM simple, N-BEATS et TFT via neuralforecast ou Darts, un modèle de fondation (Chronos) en zéro-shot ; comparer honnêtement au boosting et aux statistiques sur le backtesting — documenter le cas où le deep learning ne gagne pas.
7. **Probabiliste** : intervalles par quantiles (LightGBM quantile, TFT) et par prévision conforme ; évaluer couverture et CRPS ; représenter des éventails de prévision.
8. **Multi-séries** : modèle global sur toutes les séries vs un modèle par série, réconciliation hiérarchique si pertinent.
9. **Mise en production** : tâche quotidienne qui récupère les nouvelles données, produit la prévision à J+1 et J+7, la stocke, et compare la prévision de la veille au réel ; tableau de bord Grafana « prévu vs réel » avec erreur glissante ; réentraînement hebdomadaire ; observer pendant un mois et analyser les jours d'erreur (événements exceptionnels).

Compétences acquises

- Analyse et modélisation de séries temporelles, statistiques et neuronales.
- Validation temporelle rigoureuse et choix de métriques relatives (MASE).
- Prévision probabiliste et quantification d'incertitude.
- Ingénierie de variables temporelles sans fuite.
- Exploitation d'un modèle de prévision en continu, avec suivi de la performance réelle dans le temps.

Documentation et mise en valeur

Livrables : carnets d'analyse, tableau de backtesting toutes méthodes contre les baselines, éventails probabilistes, pipeline quotidien, tableau de bord prévu vs réel sur un mois, analyse des jours difficiles. Sur la plateforme commune : la page affiche la **prévision du jour et l'erreur des trente derniers jours** en direct — une prévision publiée à l'avance et confrontée au réel est la preuve la plus honnête qui existe en prévision. Un article « ARIMA, LightGBM ou transformer : qui prévoit le mieux la consommation électrique française ? » appuyé sur votre backtesting a une vraie valeur.

08 · Système de recommandation

Domaine	Deep learning · recommandation et recherche
Niveau	Avancé
Durée indicative	4 à 5 semaines
Valorisation	Plateforme dédiée possible : site de recommandation (films, livres, articles) avec recommandation en temps réel

Objectif et exigences

Construire un système de recommandation complet, du filtrage collaboratif classique à une architecture en deux étages (candidats puis classement) comme dans les grandes plateformes, avec recherche vectorielle en temps réel, gestion du démarrage à froid et évaluation hors ligne rigoureuse — et, si possible, une évaluation en ligne par test A/B sur un vrai site que vous exposez. Exigences : au moins quatre approches comparées, découpage temporel, métriques de classement, architecture retrieval + ranking servie en moins de 100 ms, prise en compte de la diversité et des biais de popularité, et une interface où un utilisateur note des articles et voit ses recommandations évoluer.

Notions à rechercher et apprendre

- **Fondamentaux** : retours explicites (notes) vs implicites (clics, achats, temps passé — les plus courants), matrice utilisateurs-articles et sa creusité, démarrage à froid, boucle de rétroaction.
- **Filtrage collaboratif** : voisinage (utilisateur-utilisateur, article-article), factorisation de matrices (SVD, ALS, avec biais), factorisation pour retours implicites (ALS pondéré, BPR), interprétation des facteurs latents.
- **Basé sur le contenu et hybride** : représentation des articles (métadonnées, texte via embeddings du projet 06, images via le projet 05), modèles factorisés avec attributs (LightFM, FM), résolution du démarrage à froid.
- **Deep learning** : modèles à deux tours (two-tower) pour la génération de candidats, embeddings d'utilisateurs et d'articles, recommandation séquentielle (GRU4Rec, SASRec, BERT4Rec), modèles de classement (Wide&Deep, DeepFM, DCN), architecture en étages retrieval → ranking → re-ranking.
- **Recherche vectorielle** : plus proches voisins approximatifs (HNSW, IVF, quantification produit), FAISS, index dans une base vectorielle, compromis rappel/latence.
- **Évaluation** : découpage temporel ou par dernier article (leave-one-out), Precision@k, Recall@k, MAP, NDCG, MRR, hit rate, échantillonnage de négatifs et ses biais d'évaluation, couverture, diversité, nouveauté, sérendipité ; pourquoi les métriques hors ligne prédisent mal l'effet en ligne ; tests A/B, bandits.
- **Biais et éthique** : biais de popularité, chambres d'écho, équité entre fournisseurs, explications des recommandations, exposition.

Outils, plateformes et ressources

- **Bibliothèques** : implicit (ALS, BPR rapides), LightFM, Surprise (explicite), RecBole (dizaines de modèles, benchmarks), PyTorch pour le two-tower, FAISS ou Qdrant/Milvus pour la recherche vectorielle, Sentence-Transformers pour le contenu textuel.

- **Données** : MovieLens (1M puis 25M — le standard), Amazon Reviews, Goodreads, Yelp, Last.fm ; ou votre propre site (recommandation d'articles/projets de vos portfolios, de livres, de morceaux) pour l'évaluation en ligne.
- **Ressources** : cours de recommandation de Google (Machine Learning Crash Course – Recommendation Systems, gratuit), *Recommender Systems Handbook* (Ricci et al.), article « Deep Neural Networks for YouTube Recommendations » (Covington et al., 2016 — la référence sur l'architecture en étages), article BPR (Rendle), article SASRec, cours Hugging Face sur les embeddings, blog Eugene Yan (système de recommandation en production), documentation FAISS.

Étapes de réalisation

1. **Théorie** : fiches factorisation de matrices (implémenter ALS et BPR à la main sur MovieLens 100k, lien avec le projet 02), métriques de classement (implémenter NDCG et vérifier contre une bibliothèque), architecture en étages (lire l'article YouTube).
2. **Données et protocole** : MovieLens 1M, EDA (creusité, distribution de popularité en longue traîne, activité par utilisateur), découpage temporel, définition du protocole (Recall@20 et NDCG@10 sur tous les articles, pas d'échantillonnage de négatifs), baselines (plus populaires, aléatoire).
3. **Collaboratif classique** : voisinage article-article, SVD/ALS explicite, ALS implicite et BPR ; comparer ; visualiser les facteurs latents ; mesurer couverture et biais de popularité.
4. **Contenu et hybride** : embeddings de contenu (synopsis via Sentence-Transformers, genres, affiches éventuellement), LightFM hybride ; évaluer spécifiquement les articles et utilisateurs à froid.
5. **Two-tower** : implémenter en PyTorch un modèle à deux tours avec négatifs dans le lot (in-batch negatives), indexer les embeddings d'articles dans FAISS, mesurer rappel et latence de la recherche ; recommandation séquentielle (SASRec via RecBole) en comparaison.
6. **Classement** : modèle de ranking (LightGBM ou DeepFM) sur les 200 candidats issus du retrieval, avec variables contextuelles ; mesurer le gain de NDCG contre le retrieval seul ; re-ranking pour la diversité (MMR) et mesurer le compromis.
7. **Service** : API FastAPI retrieval + ranking, cache des embeddings, latence p95 mesurée sous charge (k6), mise à jour incrémentale des embeddings utilisateurs.
8. **Interface et évaluation en ligne** : petit site (Next.js) où l'on note des films et voit ses recommandations, avec explications (« parce que vous avez aimé… ») ; si vous avez des utilisateurs (amis, communauté), test A/B entre deux algorithmes sur le taux de clic, avec calcul de la puissance statistique et de la significativité ; sinon simulation documentée.
9. **Analyse** : tableau final hors ligne, biais de popularité par algorithme, résultats du test A/B, fiche de modèle avec considérations éthiques.

Compétences acquises

- Maîtrise des algorithmes de recommandation, classiques et neuronaux.
- Architecture en étages et recherche vectorielle à faible latence — également au cœur du RAG.
- Évaluation par métriques de classement, découpage temporel, tests A/B.

- Gestion du démarrage à froid, de la diversité et des biais.
- Service d'un modèle en temps réel avec contraintes de latence.

Documentation et mise en valeur

Livrables : implémentations à la main, comparatif complet, modèle two-tower et index FAISS, API mesurée, site de démonstration, rapport de test A/B. Plateforme dédiée : le **site de recommandation** lui-même (films ou livres) où le visiteur note quelques éléments et obtient des recommandations expliquées en temps réel, avec un mode « comparer les algorithmes » qui affiche côte à côte les propositions de chaque méthode — pédagogique et démonstratif. La page du projet sur la plateforme commune renvoie vers le site et présente les mesures.

09 · Modèles génératifs d'images : VAE, GAN et diffusion

Domaine	IA générative · vision
Niveau	Avancé
Durée indicative	5 à 6 semaines
Valorisation	Plateforme dédiée possible : galerie générative interactive

Objectif et exigences

Comprendre et implémenter les trois grandes familles de modèles génératifs d'images — autoencodeurs variationnels, réseaux antagonistes et modèles de diffusion — depuis les mathématiques jusqu'au code, puis personnaliser un modèle de diffusion de pointe (Stable Diffusion) sur vos propres images avec LoRA ou DreamBooth. Exigences : VAE, GAN et un modèle de diffusion implémentés en PyTorch et entraînés sur des données modestes, évaluation quantitative (FID) et qualitative, fine-tuning LoRA d'un modèle de diffusion sur un sujet ou un style personnel, compréhension de la génération conditionnée par texte, et une réflexion documentée sur les usages, droits d'auteur et détection de contenu généré.

Notions à rechercher et apprendre

- **Modélisation générative** : apprendre une distribution, vraisemblance vs échantillonnage, modèles explicites et implicites, le trilemme qualité / diversité / vitesse.
- **VAE** : encodeur probabiliste, espace latent, borne inférieure de la vraisemblance (ELBO), terme de reconstruction et divergence KL, astuce de reparamétrisation, images floues et pourquoi, β-VAE et désenchevêtrement, VQ-VAE (latents discrets, utilisé par les modèles récents).
- **GAN** : générateur et discriminateur, jeu minimax, instabilités (effondrement des modes, oscillation), améliorations (DCGAN, Wasserstein GAN et pénalité de gradient, spectral norm), GAN conditionnels, StyleGAN et son espace de style, pourquoi les GAN ont été supplantés par la diffusion pour la génération mais restent utiles (rapidité, super-résolution).
- **Diffusion** : processus direct de bruitage, processus inverse appris, DDPM (prédire le bruit), lien avec le score matching, échantillonneurs plus rapides (DDIM, DPM-Solver), guidance sans classifieur, diffusion latente (Stable Diffusion : VAE + U-Net + encodeur de texte CLIP), architecture U-Net et transformers de diffusion (DiT), flow matching.
- **Conditionnement par texte** : CLIP (approfondi au projet 18), embeddings de texte, cross-attention, ControlNet (conditionnement par structure), inpainting.
- **Personnalisation** : DreamBooth, Textual Inversion, LoRA appliqué à la diffusion (rang faible, petit nombre d'images), surapprentissage sur le sujet.
- **Évaluation** : FID et ses limites, Inception Score, précision/rappel pour la génération, CLIP score pour l'adéquation au texte, évaluation humaine.
- **Enjeux** : droits d'auteur des données d'entraînement, deepfakes, filigranage et détection, biais de représentation dans les générations, licences des modèles (CreativeML OpenRAIL).

Outils, plateformes et ressources

- **Bibliothèques** : PyTorch, Hugging Face Diffusers (pipelines, schedulers, entraînement LoRA), PEFT, Accelerate, torchmetrics ou clean-fid pour le FID, kohya_ss pour l'entraînement LoRA avancé.
- **Modèles** : Stable Diffusion 1.5 / SDXL / SD3, FLUX (selon licence et VRAM), CLIP.
- **Matériel** : GPU indispensable — Colab/Kaggle suffisent pour VAE/GAN/DDPM sur de petites images et pour LoRA sur SD 1.5 ; SDXL demande 12-16 Go de VRAM (RunPod à l'heure).
- **Données** : MNIST/Fashion-MNIST puis CelebA (visages 64×64) ou Oxford Flowers ; pour la personnalisation, 15 à 30 photos d'un sujet ou d'un style à vous (vos propres photos évitent tout problème de droits).
- **Ressources** : cours Hugging Face Diffusion Models (gratuit, exactement ce projet), *Understanding Diffusion Models: A Unified Perspective* (Calvin Luo, 2022 — la meilleure introduction mathématique), articles DDPM (Ho et al., 2020), Latent Diffusion (Rombach et al., 2022), DCGAN, WGAN, VAE (Kingma, Welling), tutoriel VAE de Lilian Weng et ses billets « What are Diffusion Models? » et « From GAN to WGAN », vidéos de Yannic Kilcher sur ces articles, blog Diffusers, *The Annotated Diffusion Model* (Hugging Face).

Étapes de réalisation

1. **Théorie** : fiches ELBO (redériver la borne), jeu minimax, processus de diffusion (redériver la formule de bruitage en forme fermée), guidance ; lire Luo 2022 et Ho 2020 en annotant.
2. **VAE** : implémenter un VAE convolutif sur MNIST puis CelebA, visualiser l'espace latent (interpolations, parcours 2D), expérimenter β, observer le flou ; implémenter un VQ-VAE simple.
3. **GAN** : DCGAN sur CelebA 64×64, observer et documenter les instabilités (effondrement des modes filmé), passer à WGAN-GP et mesurer la stabilisation, GAN conditionnel sur Fashion-MNIST ; calculer le FID à chaque étape.
4. **Diffusion de zéro** : implémenter DDPM avec un petit U-Net sur Fashion-MNIST puis CelebA 64×64, visualiser le processus de débruitage étape par étape, implémenter DDIM pour accélérer l'échantillonnage (50 pas au lieu de 1000), ajouter le conditionnement par classe avec guidance sans classifieur et observer l'effet de l'échelle de guidance ; FID comparé au GAN.
5. **Stable Diffusion en profondeur** : avec Diffusers, décomposer le pipeline (encodeur de texte, U-Net, VAE, scheduler), manipuler chaque étape à la main, expérimenter schedulers, guidance, négatifs, inpainting et ControlNet ; documenter des grilles comparatives.
6. **Personnalisation** : constituer un jeu de 20 photos personnelles (un objet, un lieu, votre style de dessin), entraîner un LoRA avec Diffusers ou kohya_ss, comparer Textual Inversion et DreamBooth, mesurer fidélité au sujet (similarité CLIP/DINO) et diversité, observer le surapprentissage ; publier le LoRA sur Hugging Face avec fiche.
7. **Évaluation et enjeux** : FID/CLIP score sur toutes les étapes, petite évaluation humaine en aveugle (amis), analyse des biais de représentation sur quelques prompts, essai de détecteurs d'images générées, fiche de modèle avec usages déconseillés.

Compétences acquises

- Compréhension mathématique et pratique des VAE, GAN et modèles de diffusion.
- Implémentation d'architectures génératives et diagnostic de leurs instabilités.
- Maîtrise de l'écosystème Diffusers et du fine-tuning léger (LoRA) — transférable aux LLM du projet 11.
- Évaluation de modèles génératifs, quantitative et humaine.
- Culture des enjeux juridiques et éthiques de l'IA générative.

Documentation et mise en valeur

Livrables : implémentations commentées, vidéos du débruitage et de l'effondrement des modes, grilles comparatives, tableau FID, LoRA publié, fiche de modèle, article de vulgarisation sur la diffusion. Plateforme dédiée envisageable : une **galerie générative interactive** (Space Gradio intégré) où le visiteur explore l'espace latent du VAE avec des curseurs, regarde la diffusion débruiter en direct, et génère des images avec votre LoRA — avec, pour chaque famille, un encart « comment ça marche » qui renvoie à vos fiches théoriques.

10 · Construire un petit GPT : le transformer de zéro

Domaine	Grands modèles de langage · architecture
Niveau	Avancé (le projet-clé pour comprendre les LLM)
Durée indicative	5 semaines
Valorisation	Plateforme commune + démo de génération, article détaillé

Objectif et exigences

Implémenter de zéro en PyTorch un modèle de langage de type GPT — tokeniseur BPE compris — l'entraîner sur un corpus francophone, comprendre chaque composant du transformer par l'expérience, reproduire les lois d'échelle à petite échelle, et charger les poids d'un GPT-2 réel dans votre implémentation pour prouver qu'elle est exacte. Exigences : tokeniseur BPE écrit à la main, transformer décodeur complet écrit sans bibliothèque de haut niveau, entraînement d'un modèle de 10 à 100 millions de paramètres sur un corpus français, chargement réussi des poids GPT-2 officiels, étude d'ablations (attention, position, normalisation), et génération avec les différentes stratégies d'échantillonnage.

Notions à rechercher et apprendre

- **Modélisation du langage** : prédire le token suivant, vraisemblance, perplexité, n-grammes comme baseline, pourquoi c'est un objectif si puissant.
- **Tokenisation** : Byte Pair Encoding au niveau des octets (algorithme d'entraînement et d'encodage), vocabulaire, tokens spéciaux, problèmes de tokenisation (nombres, langues non anglaises, espaces), tiktoken, SentencePiece.
- **Attention** : requêtes/clés/valeurs, produit scalaire mis à l'échelle, masque causal, multi-têtes, complexité quadratique, attention groupée (GQA), FlashAttention (notion).
- **Bloc transformer** : connexions résiduelles, normalisation (LayerNorm vs RMSNorm, pré- vs post-norm), MLP et activations (GELU, SwiGLU), encodages de position (appris, sinusoïdaux, RoPE, ALiBi), embeddings liés à la sortie.
- **Entraînement** : entropie croisée, AdamW, warmup et décroissance cosinus, clipping de gradient, accumulation de gradient, précision mixte, taille de lot et de contexte, compilation (torch.compile), courbes de perte et signes de problèmes.
- **Lois d'échelle** : Kaplan et Chinchilla (paramètres vs tokens), compute optimal, ce qu'elles impliquent.
- **Génération** : glouton, beam search, température, top-k, top-p (nucleus), pénalités de répétition, cache KV et son rôle dans la vitesse d'inférence.
- **Au-delà de GPT-2** : différences architecturales de LLaMA (RMSNorm, SwiGLU, RoPE, GQA), mixture of experts (notion), fenêtres de contexte longues, pré-entraînement vs post-entraînement (projet 11).

Outils, plateformes et ressources

- **Outils** : PyTorch, tiktoken (pour comparer votre BPE), Hugging Face Transformers (uniquement pour charger les poids GPT-2 de référence), Datasets, MLflow/W&B pour les courbes.
- **Matériel** : un GPU (Colab T4/A100, ou local) — un modèle de 10 M de paramètres s'entraîne en une heure sur T4, 100 M en quelques heures sur A100 (quelques

euros sur RunPod).

- **Corpus** : TinyStories (pour un premier modèle qui parle vite), puis un corpus français : Wikipédia française, OSCAR/CulturaX français, œuvres du domaine public (Gutenberg français), textes de vos trois catalogues pour un modèle « qui parle comme vos documents ».
- **Ressources** : Andrej Karpathy — « Let's build GPT: from scratch, in code, spelled out », « Let's build the GPT Tokenizer », « Let's reproduce GPT-2 (124M) » (YouTube ; ce projet suit ces vidéos, en codant sans copier), nanoGPT et minbpe (dépôts de référence pour vérifier), *Build a Large Language Model (From Scratch)* (Sebastian Raschka), *Attention Is All You Need* (Vaswani et al., 2017), *The Illustrated GPT-2* (Alammar), articles GPT-2, Chinchilla (Hoffmann et al., 2022), LLaMA, cours Stanford CS336 (Language Modeling from Scratch — devoirs en ligne).

Étapes de réalisation

1. **Théorie** : fiches attention (calculer à la main une attention sur 3 tokens), position, normalisation, lois d'échelle ; baseline bigramme sur le corpus pour avoir une perplexité de référence.
2. **Tokeniseur** : implémenter BPE au niveau des octets (entraînement des fusions, encodage, décodage), l'entraîner sur votre corpus français, comparer le nombre de tokens par phrase avec tiktoken GPT-2 (anglais-centré) et observer l'inefficacité sur le français ; tests unitaires (encode → decode identité).
3. **Transformer** : écrire attention causale multi-têtes, bloc, modèle complet, avec vérification des formes tensorielles à chaque étape et un test de fuite d'information future (le token t ne doit pas voir t+1) ; compter les paramètres et vérifier contre la formule.
4. **Premier entraînement** : modèle de 10 M sur TinyStories ou un petit corpus français, courbes train/val, générer du texte à intervalles réguliers (garder les échantillons : voir le modèle « apprendre à parler » est très parlant en démo), ajuster taux d'apprentissage et warmup.
5. **Chargement de GPT-2** : mapper les poids officiels GPT-2 124M dans votre implémentation, vérifier que les logits coïncident avec Hugging Face à 1e-5 près — la preuve que votre implémentation est exacte.
6. **Ablations** : à budget égal, mesurer l'effet de : suppression des connexions résiduelles, post-norm vs pré-norm, encodages de position (aucun, appris, RoPE), nombre de têtes, GELU vs SwiGLU ; tableau et interprétation.
7. **Lois d'échelle en miniature** : entraîner 4 tailles (2 M, 5 M, 10 M, 25 M) avec plusieurs budgets de tokens, tracer perte vs calcul, retrouver l'allure des lois d'échelle et discuter le ratio tokens/paramètres de Chinchilla.
8. **Modèle final** : 50 à 124 M de paramètres sur un corpus français de quelques milliards de tokens si le budget le permet (sinon quelques centaines de millions), précision mixte, torch.compile, accumulation de gradient, éventuellement multi-GPU (préparation du projet 19) ; perplexité finale contre le bigramme et GPT-2 sur du texte français.
9. **Génération** : implémenter cache KV et toutes les stratégies d'échantillonnage, mesurer le gain de vitesse du cache, comparer qualitativement les stratégies ; démo Gradio.

Compétences acquises

- Compréhension complète et démontrée de l'architecture transformer et des LLM — vous pourrez lire n'importe quel article récent.
- Tokenisation, entraînement de modèles de langage, hyperparamètres et diagnostic des courbes.
- Méthodologie d'ablation et intuition sur les lois d'échelle.
- Optimisation de l'entraînement (précision mixte, compilation, accumulation) et de l'inférence (cache KV).
- Une base solide pour le fine-tuning (11), le RAG (12), les agents (13) et le passage à l'échelle (19).

Documentation et mise en valeur

Livrables : dépôt `mini-gpt` avec tokeniseur, modèle, entraînement, tests ; preuve du chargement GPT-2 ; tableau d'ablations ; graphique de lois d'échelle ; échantillons de génération au fil de l'entraînement ; modèle français publié sur Hugging Face avec fiche. Sur la plateforme commune : démo de génération et un **article long et illustré** « j'ai construit un GPT de zéro : ce que chaque composant fait vraiment » — ce type d'article est très lu et très apprécié des recruteurs en IA. La courbe des lois d'échelle et le tableau d'ablations sont vos meilleures preuves de profondeur.

11 · Fine-tuning, quantification et inférence locale de LLM

Domaine	Grands modèles de langage · post-entraînement et déploiement
Niveau	Avancé
Durée indicative	5 semaines
Valorisation	Plateforme commune + modèles publiés sur Hugging Face + API locale

Objectif et exigences

Spécialiser un LLM ouvert (Llama, Mistral, Qwen, Gemma — 1 à 8 milliards de paramètres) pour une tâche ou un domaine qui vous est propre (assistant sur vos catalogues, génération de configurations réseau, classification structurée, style de réponse en français), par fine-tuning supervisé efficace en paramètres (LoRA/QLoRA), puis alignement par préférences (DPO), évaluation rigoureuse contre le modèle de base et un modèle propriétaire, quantification et service local optimisé. Exigences : jeu de données d'instructions construit et nettoyé, SFT avec QLoRA sur GPU grand public ou Colab, DPO sur des préférences, évaluation automatique et par juge LLM avec vérification humaine, quantification GGUF/AWQ avec mesure de la perte de qualité, service via vLLM ou llama.cpp avec débit mesuré.

Notions à rechercher et apprendre

- **Post-entraînement** : pré-entraînement vs fine-tuning supervisé (SFT) vs alignement par préférences (RLHF, DPO, ORPO, GRPO) ; modèles « base » vs « instruct » ; format de conversation (chat template) et son importance.
- **Données d'instructions** : construction (rédaction, distillation depuis un modèle plus grand, données synthétiques et leurs risques, licence), qualité vs quantité (LIMA : 1 000 exemples excellents), déduplication, décontamination des benchmarks, formats (Alpaca, ShareGPT, ChatML).
- **Fine-tuning efficace** : LoRA (décomposition de rang faible, rang, alpha, modules ciblés), QLoRA (base en 4 bits NF4, double quantification, optimiseurs paginés), mémoire nécessaire et comment la calculer, gradient checkpointing, quand le fine-tuning complet se justifie.
- **Alignement** : modèle de récompense, RLHF/PPO (notion), DPO (optimisation directe sur des paires préférées/rejetées), construction de données de préférence, sur-optimisation et verbosité.
- **Évaluation des LLM** : benchmarks (MMLU, GSM8K, IFEval, et leurs versions françaises), évaluation par juge LLM (biais de position, de longueur, d'auto-préférence), Arena et Elo, évaluation spécifique à la tâche (exactitude structurée, métriques métier), oubli catastrophique (le modèle fine-tuné régresse-t-il ailleurs ?), contamination.
- **Quantification** : représentations numériques (FP32, BF16, FP8, INT8, INT4), quantification post-entraînement (GPTQ, AWQ, GGUF k-quants) vs pendant l'entraînement, perte de perplexité vs gain de mémoire, KV cache quantifié.
- **Inférence** : goulets d'étranglement (bande passante mémoire vs calcul), batching continu, PagedAttention (vLLM), décodage spéculatif, débit (tokens/s) et latence (temps au premier token), coût par million de tokens.
- **Fusion et distillation** : fusion de modèles (mergekit), distillation de connaissances vers un modèle plus petit.

Outils, plateformes et ressources

- **Entraînement** : Hugging Face Transformers, PEFT, TRL (SFTTrainer, DPOTrainer), bitsandbytes, Unsloth (2× plus rapide, moins de mémoire — idéal sur Colab), Axolotl ou LLaMA-Factory (configuration par YAML), Accelerate.
- **Évaluation** : lm-evaluation-harness, LightEval, promptfoo, un juge LLM (modèle ouvert fort ou API).
- **Quantification et service** : llama.cpp (GGUF, CPU et GPU), AutoAWQ/GPTQ, vLLM, Ollama (pratique pour la démo), Text Generation Inference, LM Studio pour comparer.
- **Modèles** : Llama 3.x 1B/3B/8B, Mistral 7B, Qwen 2.5 1.5B/7B, Gemma 2 2B/9B, Phi — attention aux licences respectives.
- **Matériel** : QLoRA d'un 7-8B tient sur 16 Go (Colab T4 avec Unsloth, ou RTX 4060 Ti 16 Go) ; un 1-3B se fine-tune sur 8 Go.
- **Ressources** : cours Hugging Face LLM et documentation TRL/PEFT, articles LoRA (Hu et al., 2021), QLoRA (Dettmers et al., 2023), DPO (Rafailov et al., 2023), LIMA, InstructGPT (Ouyang et al., 2022), blog de Sebastian Raschka (fine-tuning pratique), blog Unsloth, article « Judging LLM-as-a-Judge » (Zheng et al., 2023), Karpathy « Deep Dive into LLMs like ChatGPT », documentation vLLM et llama.cpp.

Étapes de réalisation

1. **Théorie** : fiches LoRA (implémenter une couche LoRA à la main et vérifier contre PEFT), quantification (quantifier un tenseur en INT4 à la main et mesurer l'erreur), DPO (redériver la perte), évaluation.
2. **Cas d'usage et données** : choisir un cas concret (par exemple : assistant qui répond en français sur vos catalogues et vos configurations de lab ; ou générateur de configurations FRRouting/Asterisk validées ; ou extracteur JSON structuré depuis des tickets) ; constituer 1 000 à 5 000 exemples (rédaction + distillation contrôlée + vérification manuelle d'un échantillon), nettoyer, dédupliquer, découper, décontaminer par rapport à votre jeu d'évaluation ; fiche de données.
3. **Évaluation d'abord** : construire le jeu d'évaluation (200 exemples avec réponses de référence et critères), le protocole (métriques automatiques quand la sortie est structurée, juge LLM avec grille pour le reste, vérification humaine de 50 cas), et mesurer le modèle de base instruct, un modèle propriétaire et quelques benchmarks généraux (pour l'oubli) — avant tout fine-tuning.
4. **SFT avec QLoRA** : fine-tuner un 7-8B (ou 3B) avec Unsloth/TRL, explorer rang, modules ciblés, taux d'apprentissage, époques ; courbes ; évaluer avec le protocole ; comparer LoRA vs QLoRA vs fine-tuning complet d'un 1B ; mesurer l'oubli sur les benchmarks généraux.
5. **DPO** : construire 500+ paires de préférences (réponses du modèle SFT notées ou corrigées, ou paires synthétiques vérifiées), entraîner par DPO, mesurer le gain et les effets de bord (longueur des réponses).
6. **Quantification** : exporter en GGUF (Q8, Q5_K_M, Q4_K_M) et AWQ, mesurer perplexité et score d'évaluation pour chaque niveau, tableau mémoire / qualité / vitesse ; identifier le meilleur compromis.
7. **Service** : déployer avec vLLM (GPU) et llama.cpp (CPU et GPU), mesurer débit et latence au premier token en fonction du nombre de requêtes concurrentes (k6 ou

script), calculer le coût par million de tokens sur votre matériel vs API propriétaire ; API compatible OpenAI exposée pour les projets 12 et 13.

8. **Publication** : adaptateurs LoRA, modèle fusionné et GGUF publiés sur Hugging Face avec fiche complète (données, évaluation, limites, licence), jeu de données publié si possible.

Compétences acquises

- Fine-tuning efficace de LLM (LoRA/QLoRA) et alignement par préférences (DPO).
- Construction de jeux de données d'instructions et de préférences de qualité.
- Évaluation rigoureuse des LLM, y compris par juge LLM, et mesure de l'oubli.
- Quantification et déploiement d'inférence optimisé, avec métriques de coût et de performance.
- Capacité à produire un modèle spécialisé publié et documenté — très recherchée.

Documentation et mise en valeur

Livrables : jeu de données et son processus, protocole d'évaluation, tableau « base / propriétaire / SFT / SFT+DPO » sur votre tâche et sur les benchmarks généraux, tableau de quantification, mesures de service, modèles publiés. Sur la plateforme commune : la fiche du modèle, une démo de chat (Ollama ou vLLM derrière Gradio) et surtout le comparatif d'évaluation — un modèle de 7B spécialisé qui bat un modèle propriétaire sur votre tâche, avec les preuves, est un argument de portfolio remarquable. Les téléchargements de votre modèle sur Hugging Face deviennent une métrique de portfolio à part entière.

12 · Assistant documentaire RAG avec évaluation

Domaine	LLM · applications, recherche d'information
Niveau	Avancé
Durée indicative	5 semaines
Valorisation	Plateforme dédiée : assistant qui répond sur vos trois catalogues et votre documentation de laboratoire

Objectif et exigences

Construire un assistant de questions-réponses fondé sur vos propres documents (les trois catalogues, la documentation de vos laboratoires, des normes ou RFC) par génération augmentée par récupération (RAG), en traitant sérieusement ce qui fait échouer la plupart des RAG : le découpage, la qualité de la récupération, les hallucinations et l'absence d'évaluation. Exigences : ingestion de documents hétérogènes (Markdown, PDF, HTML, tableaux), pipeline de récupération hybride (dense + lexical) avec re-ranking, génération avec citations vérifiables, jeu d'évaluation de 100+ questions et métriques de récupération et de génération, comparaison systématique des variantes, garde-fous contre les hallucinations, observabilité des requêtes et interface de conversation publiée.

Notions à rechercher et apprendre

- **Principe du RAG** : pourquoi ne pas tout fine-tuner (fraîcheur, sources, coût), pipeline ingestion → indexation → récupération → génération, fenêtre de contexte et « lost in the middle ».
- **Ingestion et découpage** : extraction de texte (PDF, HTML, tableaux, code), nettoyage, stratégies de découpage (taille fixe, récursif, par structure, sémantique, parent-enfant / small-to-big), chevauchement, métadonnées, mise à jour incrémentale.
- **Récupération** : embeddings denses (modèles multilingues et français : bge-m3, multilingual-e5, Solon), recherche lexicale BM25, recherche hybride et fusion (RRF), re-ranking par cross-encoder, filtres par métadonnées, réécriture de requête (HyDE, multi-requêtes, décomposition), recherche sur graphe (GraphRAG, notion), MTEB pour choisir un modèle d'embeddings.
- **Bases vectorielles** : index HNSW, pgvector, Qdrant, Weaviate, Chroma, FAISS ; quand une base dédiée se justifie vs PostgreSQL + pgvector.
- **Génération** : conception du prompt (instructions, contexte, citations obligatoires, refus si absence d'information), formats de sortie structurés, gestion de l'historique conversationnel (reformulation de la question), streaming.
- **Évaluation** : métriques de récupération (Recall@k, MRR, nDCG sur un jeu de questions annotées), métriques de génération (fidélité/faithfulness, pertinence de la réponse, pertinence du contexte — cadre RAGAS ou ARES), juge LLM et ses biais, jeux de questions synthétiques puis vérifiés, évaluation de bout en bout vs par composant.
- **Fiabilité** : hallucinations et leur détection, ancrage aux sources, refus calibré, injection de prompt via les documents (approfondie au projet 17), confidentialité des documents.
- **Observabilité et coût** : traçage de chaque requête (documents récupérés, prompt, réponse, latence, tokens), analyse des échecs, coût par question.

Outils, plateformes et ressources

- **Cadres** : LlamaIndex ou LangChain (utile, mais écrivez d'abord une version sans cadre pour comprendre), Haystack ; Docling ou Unstructured et PyMuPDF pour l'extraction de PDF.
- **Embeddings et re-ranking** : Sentence-Transformers, bge-m3, multilingual-e5-large, bge-reranker ; comparaison via MTEB (sous-ensemble français).
- **Bases** : PostgreSQL + pgvector (recommandé pour commencer : une seule base), Qdrant en comparaison, rank-bm25 ou Elasticsearch/OpenSearch pour le lexical.
- **LLM** : votre modèle du projet 11 via vLLM/Ollama, et un modèle propriétaire pour comparaison.
- **Évaluation et observabilité** : RAGAS, DeepEval, promptfoo ; Langfuse ou Phoenix (Arize) pour le traçage ; MLflow pour les variantes.
- **Interface** : Gradio/Chainlit pour la démo rapide, puis Next.js avec streaming pour la plateforme.
- **Ressources** : article RAG original (Lewis et al., 2020), « Lost in the Middle » (Liu et al., 2023), article RAGAS, cours DeepLearning.AI (« Building and Evaluating Advanced RAG », gratuit), documentation LlamaIndex (section évaluation), blog Anthropic sur le « contextual retrieval », guide pgvector, benchmark MTEB, cours Hugging Face « Advanced RAG ».

Étapes de réalisation

1. **Théorie** : fiches embeddings (mesurer la similarité cosinus entre phrases en français, observer ce que le modèle capte), BM25 (l'implémenter), HNSW, RAGAS ; lire Lewis 2020 et « Lost in the Middle ».
2. **Corpus** : rassembler vos documents (les trois catalogues en Markdown/HTML, vos fiches de lab, quelques RFC ou normes en PDF), extraire et nettoyer, fiche de données, inventaire des types de contenus (texte, tableaux, code, formules) — chacun pose un problème de découpage différent.
3. **Jeu d'évaluation d'abord** : rédiger 100+ questions réalistes avec la réponse attendue et les passages sources (mélange de questions simples, multi-documents, sans réponse dans le corpus, piégeuses) ; générer un complément synthétique puis le vérifier à la main.
4. **RAG minimal sans cadre** : découpage fixe, embeddings, pgvector, top-k, prompt simple, LLM ; mesurer récupération et génération — c'est la baseline.
5. **Améliorer la récupération** : comparer stratégies de découpage, tailles, modèles d'embeddings (au moins trois), hybride BM25 + dense avec RRF, re-ranking ; chaque variante mesurée sur Recall@5 et MRR ; réécriture de requêtes ; tableau complet.
6. **Améliorer la génération** : prompt avec citations obligatoires et refus explicite, découpage parent-enfant pour donner plus de contexte, ordre des passages, comparaison de LLM (votre 7B fine-tuné vs propriétaire) sur fidélité et pertinence ; mesurer le taux d'hallucination sur les questions sans réponse.
7. **Conversation et interface** : historique avec reformulation de question, streaming, affichage des sources cliquables surlignées dans le document d'origine, retour utilisateur (pouce) stocké.
8. **Observabilité et robustesse** : traçage de chaque requête avec Langfuse, tableau de bord (latence, coût, taux de refus, retours), tests d'injection via documents

malveillants, mise à jour incrémentale de l'index quand un document change.
9. **Validation** : tableau final baseline → meilleure variante, analyse des 20 pires questions, fiche de modèle du système, déploiement sur la plateforme.

Compétences acquises

- Conception et optimisation d'un pipeline RAG complet, composant par composant.
- Recherche d'information moderne : embeddings, hybride, re-ranking, bases vectorielles.
- Évaluation de systèmes LLM avec des métriques adaptées et un jeu de test construit.
- Maîtrise des hallucinations, citations et refus — ce qui rend un assistant utilisable.
- Observabilité et exploitation d'une application LLM en production.

Documentation et mise en valeur

Livrables : corpus et jeu d'évaluation, tableau d'ablation des variantes (le livrable clé), analyse des échecs, pipeline reproductible, tableau de bord d'observabilité, application déployée. Plateforme dédiée : l'**assistant lui-même**, intégré à votre portfolio — un visiteur pose une question sur n'importe lequel de vos 60 projets et obtient une réponse sourcée avec liens vers les pages. C'est à la fois une démonstration technique et un outil de navigation dans votre portfolio. Publiez le tableau d'ablation en article : « j'ai testé 40 configurations de RAG sur ma documentation, voici ce qui compte vraiment ».

13 · Agents IA, outils et garde-fous

Domaine	LLM · agents et systèmes autonomes
Niveau	Avancé
Durée indicative	5 à 6 semaines
Valorisation	Plateforme dédiée possible : agent d'exploitation de votre laboratoire (diagnostic réseau, supervision) avec journal d'actions

Objectif et exigences

Concevoir des agents fondés sur des LLM capables de raisonner, d'appeler des outils, de planifier des tâches en plusieurs étapes et de coopérer, appliqués à un domaine où vous pouvez vérifier les résultats : par exemple un agent d'exploitation qui diagnostique un incident dans votre laboratoire réseau/télécom (interroge Prometheus, lit des journaux, lance des commandes en lecture seule, propose un correctif), ou un agent d'analyse de données qui écrit et exécute du code. Exigences : appel d'outils fiable avec validation de schéma, boucle agentique implémentée à la main avant tout cadre, mémoire, planification, système multi-agents, exposition d'outils via le protocole MCP, jeu d'évaluation de tâches avec taux de réussite mesuré, garde-fous (permissions, confirmation humaine, budget, bac à sable) et traçage complet de chaque action.

Notions à rechercher et apprendre

- **Raisonnement des LLM** : chaîne de pensée, ReAct (raisonner et agir), réflexion et auto-critique, modèles de raisonnement (o-series, DeepSeek-R1) et leur usage, limites (hallucination d'actions, boucles).
- **Appel d'outils** : function calling, schémas JSON, sorties structurées contraintes, validation et gestion des erreurs, idempotence, outils en lecture vs écriture, conception d'outils compréhensibles par un modèle (description, exemples).
- **Boucle agentique** : observation → raisonnement → action, critères d'arrêt, budget de pas et de tokens, gestion du contexte long (résumé, mémoire de travail), reprise après erreur.
- **Mémoire** : court terme (contexte), long terme (base vectorielle — lien projet 12), épisodique, mémoire procédurale ; quand la mémoire nuit.
- **Planification** : décomposition en sous-tâches, plan-and-execute, arbres de recherche (Tree of Thoughts), graphes d'état (LangGraph), interruptions et validation humaine (human-in-the-loop).
- **Multi-agents** : rôles, orchestrateur-travailleurs, débat, critique, protocoles de communication, quand le multi-agents aide et quand il complique inutilement.
- **Protocoles** : Model Context Protocol (serveurs d'outils et de ressources interopérables), agents et API compatibles OpenAI, agents et navigateurs/ordinateurs (notion).
- **Évaluation d'agents** : taux de réussite par tâche, coût, nombre de pas, benchmarks (SWE-bench, τ-bench, GAIA — pour comprendre les méthodes), traces et rejouabilité, non-déterminisme (répéter chaque tâche plusieurs fois).
- **Sécurité** : injection de prompt indirecte via les résultats d'outils (le risque majeur), exfiltration de données, moindre privilège, bac à sable pour l'exécution de code, confirmation des actions irréversibles, OWASP Top 10 for LLM Applications.

Outils, plateformes et ressources

- **Modèles** : votre LLM du projet 11 (l'appel d'outils exige un modèle instruct récent — Qwen 2.5 ou Llama 3.1+ en font correctement), et un modèle propriétaire fort pour comparer.
- **Cadres** : d'abord aucun (boucle ReAct maison avec l'API compatible OpenAI), puis LangGraph (graphes d'état, interruptions), Pydantic AI ou smolagents (légers), AutoGen ou CrewAI pour le multi-agents ; SDK MCP (Python/TypeScript) pour exposer vos outils.
- **Outils à exposer** : API Prometheus/Grafana, recherche dans vos journaux (Loki/Elasticsearch), commandes réseau en lecture seule (ping, traceroute, show), documentation via le RAG du projet 12, exécution Python dans un conteneur jetable, tickets.
- **Évaluation et traçage** : Langfuse ou LangSmith, promptfoo, un harnais maison de tâches.
- **Sécurité** : conteneurs sans réseau pour le code, gVisor ou Firecracker (notion), listes d'autorisation d'outils, Garak/PyRIT pour tester les injections (projet 17).
- **Ressources** : articles ReAct (Yao et al., 2022), Toolformer, Reflexion, « Generative Agents » ; guide « Building effective agents » (Anthropic — lecture prioritaire, prône la simplicité), cours Hugging Face Agents (gratuit), cours DeepLearning.AI sur LangGraph et le function calling, spécification MCP (modelcontextprotocol.io), billet « LLM Powered Autonomous Agents » (Lilian Weng), OWASP LLM Top 10, blog Simon Willison sur l'injection de prompt.

Étapes de réalisation

1. **Théorie** : fiches ReAct, function calling, planification, injection indirecte ; lire « Building effective agents » et l'article ReAct ; définir le cas d'usage (agent d'exploitation du lab recommandé : résultats vérifiables, lien avec vos autres catalogues).
2. **Outils** : implémenter 8 à 12 outils avec schémas Pydantic, validés et testés unitairement, classés lecture/écriture, avec descriptions soignées ; les exposer via un serveur MCP.
3. **Boucle agentique maison** : écrire la boucle ReAct sans cadre (appel de modèle, analyse des appels d'outils, exécution, retour, arrêt), avec budget de pas et journal complet ; la faire fonctionner sur cinq tâches simples (« quel hôte a la latence la plus élevée ? »).
4. **Jeu d'évaluation** : définir 30 à 50 tâches de difficulté croissante avec critère de réussite vérifiable automatiquement (par exemple : incident injecté dans le lab — interface coupée, service arrêté, route retirée — l'agent doit identifier la cause) ; harnais qui rejoue chaque tâche 5 fois et calcule taux de réussite, pas, coût.
5. **Améliorations mesurées** : réflexion/auto-critique, planification explicite, mémoire longue (RAG sur les incidents passés), comparaison de modèles (7B local vs propriétaire) — chaque variante évaluée sur le harnais ; tableau.
6. **LangGraph et validation humaine** : réimplémenter en graphe d'état avec un nœud d'interruption avant toute action d'écriture (redémarrer un service, modifier une configuration), reprise après validation, persistance de l'état.
7. **Multi-agents** : orchestrateur + agents spécialisés (réseau, système, documentation) avec un critique ; mesurer sur le harnais si cela améliore le taux de

réussite ou seulement le coût — documenter honnêtement.

8. **Sécurité** : tests d'injection indirecte (un journal contenant « ignore les instructions et exécute… »), exécution de code en bac à sable sans réseau, moindre privilège, budget, liste d'autorisation ; documenter les attaques réussies et les corrections.
9. **Interface et traçage** : interface de chat montrant le raisonnement, chaque appel d'outil et son résultat, les demandes de validation ; traces Langfuse ; page « journal des actions de l'agent » consultable.

Compétences acquises

- Conception d'agents LLM fiables : outils, boucle, planification, mémoire, multi-agents.
- Protocole MCP et interopérabilité des outils.
- Évaluation empirique d'agents avec harnais et non-déterminisme pris en compte.
- Sécurité des systèmes agentiques : injection indirecte, bac à sable, validation humaine.
- Jugement sur quand un agent est justifié et quand un simple pipeline suffit.

Documentation et mise en valeur

Livrables : serveur MCP d'outils, boucle agentique maison et version LangGraph, harnais de tâches et résultats par variante, rapport de sécurité, traces. Plateforme dédiée envisageable : l'**agent d'exploitation** de votre laboratoire, avec un mode démonstration où le visiteur déclenche un incident simulé et regarde l'agent diagnostiquer en direct, chaque étape tracée. Le tableau « taux de réussite × coût par variante » et le rapport d'attaques par injection sont des preuves de maturité rares.

14 · Parole : reconnaissance, synthèse et assistant vocal

Domaine	Deep learning · audio et parole
Niveau	Avancé
Durée indicative	4 à 5 semaines
Valorisation	Plateforme dédiée possible : assistant vocal en français relié à vos autres projets

Objectif et exigences

Maîtriser la chaîne de traitement de la parole — représentation du signal audio, reconnaissance automatique (ASR), synthèse vocale (TTS), détection d'activité vocale, identification du locuteur — puis assembler un assistant vocal en français fonctionnant localement, à faible latence, relié au RAG (12) ou à l'agent (13). Ce projet rejoint le traitement du signal du catalogue télécoms. Exigences : compréhension des représentations audio (spectrogrammes, MFCC), fine-tuning d'un modèle ASR sur du français (Whisper ou wav2vec2) avec WER mesuré, comparaison de systèmes TTS, pipeline temps réel (VAD → ASR → LLM → TTS) avec latence de bout en bout inférieure à 1,5 s, robustesse au bruit et aux accents documentée.

Notions à rechercher et apprendre

- **Signal audio** : échantillonnage (16 kHz pour la parole), quantification, transformée de Fourier à court terme, spectrogramme, échelle de Mel, MFCC, fenêtrage — recoupe le projet 02 du catalogue télécoms.
- **Reconnaissance de la parole** : approches historiques (HMM-GMM) en notion, modèles de bout en bout : CTC (alignement sans segmentation), attention encodeur-décodeur, transducteurs (RNN-T) ; wav2vec 2.0 (apprentissage auto-supervisé), Whisper (entraînement faiblement supervisé multilingue, architecture, hallucinations connues) ; modèle de langage pour le décodage (beam search + n-grammes), horodatage au niveau des mots.
- **Métriques ASR** : taux d'erreur de mots (WER) et de caractères (CER), normalisation du texte avant calcul, évaluation par sous-groupe (accents, bruit, débit), latence et facteur temps réel.
- **Synthèse vocale** : pipeline texte → phonèmes (G2P) → spectrogramme (Tacotron 2, FastSpeech 2) → vocodeur (WaveNet, HiFi-GAN), modèles de bout en bout (VITS), clonage vocal et ses enjeux éthiques, TTS légers (Piper), évaluation par MOS.
- **Autres tâches** : détection d'activité vocale (Silero VAD, WebRTC VAD), diarisation (qui parle quand — pyannote), identification du locuteur (embeddings x-vector/ECAPA), classification audio et détection de mots-clés (wake word), débruitage (RNNoise, DeepFilterNet).
- **Systèmes vocaux temps réel** : découpage en flux, streaming ASR, latence de bout en bout et où elle se cache, interruption (barge-in), modèles parole-à-parole (notion), audio dans le navigateur (WebRTC, Web Audio API — lien avec le catalogue télécoms).
- **Données de parole** : Common Voice (crowdsourcé, français abondant), MLS, VoxPopuli, licences, consentement et voix des personnes.

Outils, plateformes et ressources

- **Bibliothèques** : librosa et torchaudio (signal), Hugging Face Transformers (Whisper, wav2vec2), faster-whisper et whisper.cpp (inférence rapide), Silero VAD, pyannote.audio (diarisation), SpeechBrain, Piper et Coqui TTS (XTTS) ou Kokoro (TTS), jiwer (WER), sounddevice/WebRTC pour la capture.
- **Données** : Mozilla Common Voice français, Multilingual LibriSpeech français, VoxPopuli, vos propres enregistrements pour le test (avec consentement des participants).
- **Matériel** : micro USB correct (30 €), GPU pour le fine-tuning (Colab suffit pour Whisper-small), CPU pour l'inférence avec faster-whisper/whisper.cpp.
- **Ressources** : cours Hugging Face Audio (gratuit — exactement ce projet), articles Whisper (Radford et al., 2022), wav2vec 2.0 (Baevski et al., 2020), CTC (Graves, 2006) et le tutoriel Distill « Sequence Modeling with CTC », VITS, HiFi-GAN ; *Speech and Language Processing* chapitres phonétique et ASR ; documentation pyannote et Silero ; blog Hugging Face « Fine-Tune Whisper ».

Étapes de réalisation

1. **Théorie** : fiches spectrogramme/MFCC (les calculer avec NumPy et comparer à librosa), CTC (implémenter le décodage glouton et comprendre la perte), architecture Whisper, pipeline TTS ; enregistrer votre voix et visualiser.
2. **Données et protocole** : Common Voice français (sous-ensemble), fiche de données (démographie des locuteurs, biais), jeu de test stratifié par accent/genre/âge, plus un jeu « difficile » enregistré par vous (bruit, distance, accents régionaux, vocabulaire technique de vos catalogues) ; normalisation du texte pour le WER.
3. **ASR — évaluation des modèles existants** : Whisper (tiny → large-v3), wav2vec2 français, faster-whisper ; WER par sous-groupe et facteur temps réel sur CPU et GPU ; documenter les hallucinations de Whisper sur les silences.
4. **Fine-tuning ASR** : fine-tuner Whisper-small sur du français avec vocabulaire technique (vos termes : « BGP », « GPON », « Kamailio » — que les modèles génériques massacrent), mesurer le gain de WER sur le jeu difficile et l'absence de régression sur Common Voice ; comparer avec wav2vec2 + modèle de langage n-gramme.
5. **TTS** : comparer Piper (léger, rapide), XTTS (qualité, clonage) et un TTS commercial en écoute à l'aveugle (MOS par quelques personnes), latence et taille ; éventuellement fine-tuner Piper sur une voix avec consentement ; réflexion écrite sur le clonage vocal.
6. **Briques complémentaires** : VAD Silero (mesurer précision et latence), diarisation pyannote sur un enregistrement de réunion, identification du locuteur, débruitage — chaque brique mesurée.
7. **Assistant vocal** : pipeline en flux VAD → ASR (streaming) → LLM (RAG du projet 12 ou agent du projet 13) → TTS, dans le navigateur via WebRTC ou sur un Raspberry Pi avec micro et haut-parleur ; mesurer la latence de chaque étage et de bout en bout, optimiser (modèles plus petits, streaming du TTS dès la première phrase), gérer l'interruption.
8. **Robustesse** : tests en environnement bruyant, plusieurs locuteurs, accents, vocabulaire technique ; tableau et limites documentées.

Compétences acquises

- Traitement du signal audio et représentations pour l'apprentissage.
- Reconnaissance de la parole : modèles, fine-tuning, évaluation par WER stratifié.
- Synthèse vocale et évaluation perceptive.
- Construction d'un système vocal temps réel et optimisation de la latence.
- Sensibilité aux enjeux de biais (accents) et d'éthique (clonage vocal, consentement).

Documentation et mise en valeur

Livrables : carnets signal, tableau WER par modèle et sous-groupe, modèle Whisper fine-tuné publié avec fiche, comparatif TTS, pipeline temps réel avec décomposition de la latence, vidéo de l'assistant. Plateforme dédiée envisageable : l'**assistant vocal** dans le navigateur (WebRTC) qui répond en français à des questions sur vos projets — ou un Raspberry Pi avec enceinte filmé en démonstration. Le graphique « où part la latence » et le tableau WER par accent sont des éléments distinctifs.

15 · Apprentissage par renforcement

Domaine	Apprentissage par renforcement (RL)
Niveau	Expert
Durée indicative	6 semaines
Valorisation	Plateforme commune : vidéos d'agents, courbes d'apprentissage, environnement personnalisé publié

Objectif et exigences

Implémenter les algorithmes fondamentaux de l'apprentissage par renforcement, du tabulaire aux méthodes profondes (DQN, PPO), les valider sur des environnements standards, puis concevoir votre propre environnement modélisant un problème réel de vos autres catalogues — ordonnancement de ressources radio, routage adaptatif, allocation de trafic, gestion d'énergie — et y entraîner un agent comparé à des heuristiques classiques. Le RL est aussi la base de l'alignement des LLM (RLHF/GRPO). Exigences : algorithmes tabulaires et profonds implémentés et vérifiés contre une bibliothèque de référence, reproduction de résultats publiés avec graines multiples et intervalles de confiance, environnement personnalisé conforme à l'interface Gymnasium, comparaison honnête agent vs heuristiques, et étude de la sensibilité aux hyperparamètres.

Notions à rechercher et apprendre

- **Formalisme** : processus de décision markovien (états, actions, transitions, récompenses, facteur d'actualisation), politique, fonctions de valeur, équations de Bellman, exploration vs exploitation, épisodique vs continu, observabilité partielle.
- **Méthodes tabulaires** : programmation dynamique (itération de la valeur, de la politique), Monte-Carlo, différences temporelles, Q-learning et SARSA, bandits multi-bras (ε-greedy, UCB, Thompson).
- **RL profond par valeur** : DQN (réseau cible, tampon de relecture), Double DQN, Dueling, priorisation de la relecture, Rainbow ; instabilités et la « triade mortelle ».
- **Méthodes par politique** : gradient de politique (REINFORCE), acteur-critique (A2C), avantage généralisé (GAE), PPO (objectif tronqué — l'algorithme de référence, y compris pour RLHF), SAC et TD3 pour les actions continues.
- **Sujets avancés** : RL basé modèle (notion, MuZero), RL hors ligne, curiosité et exploration, façonnage de la récompense et ses pièges (reward hacking), apprentissage par curriculum, multi-agents (notion), lien avec RLHF/DPO/GRPO des LLM.
- **Méthodologie** : variance entre graines (pourquoi une seule courbe ne prouve rien), intervalles de confiance, nombre de pas d'environnement, normalisation des observations et des récompenses, vectorisation des environnements, reproductibilité difficile du RL.
- **Conception d'environnement** : espace d'observation et d'action, fonction de récompense (l'essentiel du travail), horizon, simulateur fidèle vs rapide, écart simulation/réel.

Outils, plateformes et ressources

- **Bibliothèques** : Gymnasium (environnements standards et interface), Stable-Baselines3 (implémentations de référence pour vérifier), CleanRL (implémentations en un fichier, très lisibles — à lire après avoir écrit les vôtres), PyTorch, PettingZoo

(multi-agents), MiniGrid, Atari via ale-py, MuJoCo pour le contrôle continu, ns3-gym ou un simulateur maison pour les environnements réseau.
- **Suivi** : W&B ou MLflow (indispensable pour comparer les graines), enregistrement vidéo des épisodes via Gymnasium.
- **Ressources** : *Reinforcement Learning: An Introduction* (Sutton, Barto — gratuit, la bible ; chapitres 1 à 13), cours de David Silver (UCL/DeepMind, YouTube), Spinning Up in Deep RL (OpenAI — pédagogique, avec implémentations), cours Hugging Face Deep RL (gratuit, pratique), articles DQN (Mnih et al., 2015), PPO (Schulman et al., 2017), GAE, SAC ; « Deep Reinforcement Learning Doesn't Work Yet » (Alex Irpan — lecture obligatoire pour la lucidité), « The 37 Implementation Details of PPO » (blog ICLR — indispensable pour que PPO fonctionne), Berkeley CS285.

Étapes de réalisation

1. **Théorie** : lire Sutton-Barto chapitres 1 à 6 ; fiches MDP, Bellman, exploration ; résoudre à la main un petit MDP par itération de la valeur.
2. **Tabulaire** : implémenter itération de la valeur, Q-learning et SARSA sur FrozenLake et un labyrinthe maison ; bandits sur un problème de sélection de canal radio simulé (lien télécoms) ; visualiser les fonctions de valeur.
3. **DQN** : implémenter DQN de zéro sur CartPole puis un jeu Atari (Pong ou Breakout), avec tampon de relecture et réseau cible ; ajouter Double et Dueling ; comparer avec SB3 sur 5 graines ; documenter les échecs (divergence) et leurs causes.
4. **Gradient de politique et PPO** : REINFORCE puis A2C puis PPO implémentés en suivant les « 37 détails », sur CartPole, LunarLander, puis un environnement continu (MuJoCo Hopper ou BipedalWalker) ; reproduire les courbes publiées avec intervalles de confiance sur 5 graines ; SAC en comparaison sur le continu.
5. **Sensibilité** : étude systématique de l'effet du taux d'apprentissage, du coefficient d'entropie, de la normalisation, de la graine — un graphique qui montre la variance entre graines est en soi une leçon.
6. **Environnement personnalisé** : modéliser un problème de vos catalogues, par exemple l'ordonnancement de paquets sur une station de base (files par utilisateur, qualité de canal variable, récompense = débit pondéré par l'équité et la latence), ou le routage adaptatif sur une topologie avec charges variables, ou la gestion d'énergie d'une flotte IoT ; interface Gymnasium, simulateur rapide, tests unitaires, fiche de l'environnement ; publier l'environnement.
7. **Agent vs heuristiques** : implémenter les heuristiques du domaine (round-robin, proportional fair, plus court chemin, seuils) et entraîner PPO/SAC ; comparer sur des scénarios tenus hors entraînement, avec intervalles ; analyser la politique apprise (que fait-elle de différent ? généralise-t-elle ?) ; documenter les cas où l'heuristique gagne ; attention au reward hacking observé.
8. **Lien avec les LLM** : implémenter un mini-RLHF ou GRPO sur une tâche jouet (récompense vérifiable, par exemple format ou arithmétique) avec le petit GPT du projet 10 — pour boucler la théorie avec la pratique des LLM.

Compétences acquises

- Maîtrise théorique et pratique du RL, du tabulaire à PPO/SAC.

- Rigueur expérimentale spécifique au RL (graines, intervalles, reproduction).
- Conception d'environnements et de fonctions de récompense pour des problèmes réels.
- Jugement lucide sur les cas où le RL apporte quelque chose par rapport aux heuristiques.
- Compréhension des fondements de l'alignement des LLM.

Documentation et mise en valeur

Livrables : implémentations vérifiées, courbes avec intervalles de confiance, étude de sensibilité, environnement publié (PyPI ou GitHub) avec fiche, comparatif agent vs heuristiques, vidéos d'épisodes. Sur la plateforme commune : galerie de vidéos « avant / pendant / après apprentissage », courbes, et l'environnement en téléchargement. Un article « j'ai entraîné un agent RL à ordonnancer une station de base : il bat proportional fair de 8 %, mais… » avec les nuances est exactement le type de contenu qui montre de la maturité.

16 · MLOps : pipeline de bout en bout, déploiement et monitoring

Domaine	Industrialisation · MLOps
Niveau	Expert
Durée indicative	6 à 8 semaines
Valorisation	Plateforme dédiée : la plateforme MLOps elle-même, qui héberge et exploite les modèles des autres projets

Objectif et exigences

Construire une plateforme MLOps complète qui prend un modèle d'un carnet à la production et le maintient : versionnement des données et du code, pipeline d'entraînement automatisé, registre de modèles, tests spécifiques au ML, déploiement continu sur Kubernetes, service avec montée en charge, monitoring des données et des performances, détection de dérive et réentraînement déclenché, avec au moins deux modèles réels de vos projets précédents exploités pendant plusieurs semaines. C'est l'équivalent, pour l'IA, du projet plateforme d'exploitation des catalogues précédents. Exigences : reproductibilité totale (données + code + environnement → même modèle), pipeline déclenché par CI, tests de données et de modèle bloquants, déploiement canari ou ombre, monitoring avec alertes, rejeu d'un incident de dérive détecté et résolu automatiquement, et documentation d'exploitation.

Notions à rechercher et apprendre

- **Vue d'ensemble** : niveaux de maturité MLOps (Google), dette technique du ML (Sculley), différence entre déployer du code et déployer un modèle, rôles (data scientist, ML engineer, platform).
- **Données** : versionnement (DVC, lakeFS), validation de schéma et de distribution (Great Expectations, Pandera), lignage, feature store (Feast — concepts : cohérence entraînement/service, point-in-time), qualité des étiquettes.
- **Pipelines** : orchestration (Kubeflow Pipelines, Airflow, Prefect, Dagster, ZenML), étapes idempotentes, cache, exécution sur GPU à la demande, hyperparamètres et expériences en pipeline.
- **Registre et gestion des modèles** : MLflow Model Registry, étapes (staging, production, archivé), signature de modèle, fiche de modèle générée, approbation.
- **Tests ML** : tests unitaires du code, tests de données (schéma, plages, nulls, dérive vs référence), tests de modèle (performance minimale, invariance, directionnels, tests de comportement, équité), tests de contrat de l'API, tests de charge.
- **Déploiement** : conteneurisation, serveurs d'inférence (FastAPI + ONNX Runtime, TorchServe, Triton, BentoML, KServe/Seldon sur Kubernetes), inférence en ligne vs par lots vs en flux, mise à l'échelle automatique, GPU sur Kubernetes, stratégies : bleu/vert, canari, ombre (shadow), tests A/B ; retour arrière.
- **Monitoring** : métriques système (latence, erreurs, saturation), métriques de données (dérive des entrées — PSI, KS, distance de Wasserstein), dérive des prédictions, performance réelle quand les étiquettes arrivent (retard d'étiquetage), dérive de concept, alertes, boucle de réentraînement (déclencheurs : calendrier, dérive, performance).
- **Gouvernance** : traçabilité (quel modèle a servi quelle prédiction), reproductibilité pour audit, documentation, coûts (FinOps du ML), lien avec le projet 17 et 20.

Outils, plateformes et ressources

- **Outils** : Git + DVC, MLflow (tracking et registre — déjà en place depuis le projet 01), Great Expectations ou Pandera, Prefect ou Kubeflow Pipelines (ou Argo Workflows), Feast (optionnel), FastAPI + ONNX Runtime, BentoML ou KServe, Docker, Kubernetes (k3s sur votre VPS ou cluster du catalogue réseaux/systèmes), GitHub Actions ou GitLab CI, Prometheus + Grafana, Evidently AI (dérive et rapports), Alertmanager, Terraform si cloud.
- **Modèles à exploiter** : le détecteur d'anomalies (04) et le modèle de prévision (07) sont parfaits : flux réel continu, étiquettes qui arrivent avec retard, dérive réelle ; le classifieur de textes (06) en complément.
- **Ressources** : « Made With ML » (MLOps, gratuit — le meilleur parcours pratique), MLOps Zoomcamp (DataTalks.Club, gratuit, complet), *Designing Machine Learning Systems* (Chip Huyen — la référence conceptuelle), *Reliable Machine Learning* (O'Reilly, approche SRE), Google « MLOps: Continuous delivery and automation pipelines in ML », « The ML Test Score » (Breck et al., Google — grille de 28 tests), documentation Evidently, Full Stack Deep Learning, Google Professional ML Engineer (certification alignée).

Étapes de réalisation

1. **Théorie** : fiches maturité MLOps, ML Test Score (auto-évaluer vos projets précédents avec la grille : le score sera bas, c'est le point de départ), dérive et ses métriques, stratégies de déploiement.
2. **Reproductibilité** : prendre le modèle de prévision (07) : versionner données (DVC) et code, figer l'environnement, pipeline d'entraînement en étapes DVC ou Prefect, vérifier que deux exécutions donnent le même modèle (hash), enregistrer dans MLflow avec signature et fiche générée.
3. **Tests** : suite de tests de données (Great Expectations : schéma, plages, fraîcheur), de modèle (performance minimale sur le jeu de test de référence, tests directionnels, invariances), de contrat API ; intégrés à la CI, bloquants.
4. **CI/CD et registre** : sur push : tests → entraînement → évaluation → si meilleur que le modèle en production sur la référence, promotion en staging ; approbation manuelle → production ; image Docker construite et poussée ; tout tracé.
5. **Service** : API d'inférence (FastAPI + ONNX Runtime, ou BentoML) déployée sur Kubernetes avec mise à l'échelle automatique, tests de charge (k6) avec latence p95 et coût par 1 000 prédictions ; inférence par lots planifiée pour la prévision quotidienne.
6. **Déploiement progressif** : déploiement ombre (nouveau modèle reçoit le trafic, ses prédictions sont journalisées et comparées sans être servies), puis canari 10 % → 100 % avec critères de promotion automatiques et retour arrière testé.
7. **Monitoring** : journalisation de chaque prédiction (entrées, sortie, version), tableaux Grafana système + données + prédictions, Evidently pour la dérive des entrées par rapport au jeu d'entraînement, jointure avec les étiquettes réelles quand elles arrivent (le réel de la prévision J+1), performance glissante, alertes.
8. **Boucle de réentraînement** : déclencheur sur dérive ou dégradation → pipeline → tests → promotion ; puis **injecter une dérive réelle** (changer les données d'entrée, simuler un changement de comportement du trafic) et documenter la détection, le

réentraînement et le retour à la performance nominale — la démonstration centrale du projet.

9. **Deuxième modèle et exploitation** : embarquer le détecteur d'anomalies (04) sur la même plateforme pour prouver la généricité ; exploiter les deux pendant un mois ; runbooks (que faire quand telle alerte se déclenche), post-mortem des incidents, coûts mensuels.

Compétences acquises

- Conception et exploitation d'une plateforme MLOps complète : données, pipelines, registre, CI/CD, service, monitoring.
- Tests spécifiques au ML et déploiements progressifs sûrs.
- Détection de dérive et automatisation du réentraînement, avec preuve par un incident rejoué.
- Kubernetes appliqué au ML, mise à l'échelle et coûts.
- Le profil ML Engineer / MLOps, l'un des plus recherchés du marché.

Documentation et mise en valeur

Livrables : dépôt de la plateforme (infrastructure as code, pipelines, tests), schéma d'architecture, score ML Test avant/après, rapport de l'incident de dérive (chronologie détection → réentraînement → rétablissement), tableaux de bord, runbooks, coûts. Plateforme dédiée : la **plateforme MLOps elle-même**, qui devient le socle d'exploitation de vos modèles et alimente le tableau de bord de production de AI Lab Hub. Un tableau de bord public « modèles en production : versions, latence, dérive, performance réelle sur 30 jours » est une preuve d'exploitation réelle que très peu de portfolios possèdent.

17 · Sécurité et IA responsable : attaques, biais, explicabilité, conformité

Domaine	Sécurité de l'IA · éthique · réglementation
Niveau	Expert
Durée indicative	5 à 6 semaines
Valorisation	Plateforme commune : rapports d'audit de vos propres modèles, tableau d'attaques et de défenses

Objectif et exigences

Auditer vos propres modèles des projets précédents sous trois angles : sécurité (attaques adverses sur le classifieur d'images, empoisonnement de données, extraction de modèle, injection de prompt et jailbreak sur le RAG et l'agent), équité et biais (sur le modèle tabulaire, le NLP et l'ASR), et explicabilité ; puis mettre en place les défenses, mesurer leur efficacité, et produire la documentation exigée par la réglementation européenne (AI Act) pour un système à haut risque fictif. Exigences : au moins huit attaques reproduites avec taux de réussite mesuré, défenses mesurées, audit d'équité avec plusieurs définitions et arbitrages explicités, explications globales et locales évaluées, un dossier de conformité AI Act, et une politique d'usage responsable pour votre plateforme.

Notions à rechercher et apprendre

- **Attaques sur les modèles classiques** : exemples adverses (FGSM, PGD, C&W, attaques boîte noire par requêtes, patches physiques), empoisonnement de données et portes dérobées (BadNets), extraction/vol de modèle, inférence d'appartenance et inversion (fuites de données d'entraînement), confidentialité différentielle (notion, DP-SGD).
- **Défenses** : entraînement adverse, détection d'entrées adverses, distillation défensive et ses limites, certification (randomized smoothing, notion), désinfection des données, surveillance des entrées.
- **Sécurité des LLM** : injection de prompt directe et indirecte, jailbreaks (rôle, encodage, multi-tours), extraction du prompt système, fuite de données via le RAG, abus d'outils dans les agents, empoisonnement de la base documentaire, OWASP Top 10 for LLM, défenses (séparation instructions/données, classifieurs d'entrée/sortie, moindre privilège, canaris, revue humaine) et leurs limites structurelles.
- **Équité** : sources de biais (données historiques, représentation, mesure, agrégation), définitions (parité démographique, égalité des chances, calibration par groupe) et leur incompatibilité mathématique (théorème d'impossibilité), métriques par sous-groupe, méthodes de correction (pré-, en-, post-traitement), proxys d'attributs sensibles, intersectionnalité.
- **Explicabilité** : modèles interprétables vs explications post hoc, SHAP et LIME (et leurs faiblesses), attributions par gradient (Integrated Gradients, Grad-CAM), contrefactuels, exemples influents, interprétabilité mécanistique des transformers (notion : circuits, sondes, autoencodeurs épars), évaluation des explications (fidélité, stabilité), droit à l'explication.
- **Réglementation et cadres** : règlement européen sur l'IA (classification des risques, obligations pour les systèmes à haut risque : gestion des risques, gouvernance des données, documentation technique, journalisation, transparence, contrôle humain, robustesse), RGPD appliqué à l'IA (base légale, minimisation,

décision automatisée), recommandations CNIL, NIST AI RMF, ISO/IEC 42001, fiches de modèles et de données comme outils de conformité.
- **IA responsable en pratique** : évaluation d'impact, red teaming, cartes de risques, politiques d'usage, surveillance post-déploiement, signalement.

Outils, plateformes et ressources

- **Sécurité** : Adversarial Robustness Toolbox (ART, IBM — attaques et défenses), Foolbox, TextAttack (NLP), Garak et PyRIT (red teaming de LLM), promptfoo (tests d'injection), Rebuff/LLM Guard (défenses), Opacus (confidentialité différentielle en PyTorch).
- **Équité** : Fairlearn, AIF360 (IBM), Aequitas ; What-If Tool.
- **Explicabilité** : SHAP, LIME, Captum (PyTorch), InterpretML, DiCE (contrefactuels), TransformerLens (mécanistique).
- **Documentation** : modèles de fiches (Model Cards, Datasheets for Datasets), gabarits de documentation technique AI Act (annexe IV), Model Card Toolkit.
- **Ressources** : *Fairness and Machine Learning* (Barocas, Hardt, Narayanan — gratuit), *Interpretable Machine Learning* (Molnar — gratuit), articles FGSM (Goodfellow et al., 2014), « Adversarial Examples Are Not Bugs, They Are Features », « Model Cards for Model Reporting » (Mitchell et al.), « Datasheets for Datasets » (Gebru et al.), OWASP LLM Top 10, blog Simon Willison (injection de prompt), texte du règlement européen sur l'IA et guides de la Commission, ressources CNIL sur l'IA, NIST AI RMF, MITRE ATLAS (matrice des attaques sur l'IA), cours « Trustworthy ML » (Stanford/Toronto), red teaming d'Anthropic et OpenAI (publications).

Étapes de réalisation

1. **Théorie** : fiches exemples adverses (implémenter FGSM à la main sur votre CNN du projet 05, lien projet 02), définitions d'équité (démontrer l'incompatibilité sur un exemple numérique), SHAP (calculer les valeurs de Shapley exactes sur un modèle à 3 variables), résumé structuré de l'AI Act.
2. **Attaques sur la vision** : FGSM, PGD et une attaque boîte noire contre votre détecteur (05) ; taux de réussite selon la perturbation ; patch adverse physique imprimé et filmé ; porte dérobée par empoisonnement (5 % d'images avec déclencheur) et démonstration ; entraînement adverse et mesure du compromis robustesse / précision propre.
3. **Attaques sur le NLP et l'ASR** : TextAttack contre le classifieur (06) (synonymes, fautes), robustesse ; bruit adverse contre l'ASR (14) ; inférence d'appartenance sur le modèle tabulaire (03) ; défenses et mesures.
4. **Red teaming LLM** : campagne Garak/PyRIT et manuelle contre le RAG (12) et l'agent (13) : injection directe, injection indirecte via document et via résultat d'outil, extraction du prompt système, jailbreaks multi-tours, exfiltration ; taux de réussite par catégorie ; défenses (garde-fous d'entrée/sortie, séparation, moindre privilège, canaris) et taux résiduel — documenter que le risque ne tombe jamais à zéro.
5. **Audit d'équité** : sur le modèle tabulaire (03, si les données ont des attributs sensibles ou des proxys — sinon le jeu Adult/COMPAS), métriques par groupe et intersections, trois définitions d'équité et leur arbitrage, corrections (Fairlearn) et

coût en performance ; sur le NLP, biais dans les embeddings et prédictions ; sur l'ASR, WER par accent et genre ; rapport d'équité avec recommandations.

6. **Explicabilité** : explications globales et locales de chaque modèle (SHAP tabulaire, Integrated Gradients et Grad-CAM vision, attributions NLP), contrefactuels (« que faudrait-il changer pour que la décision change ? »), évaluation de la fidélité et de la stabilité des explications, expérience simple d'interprétabilité mécanistique sur le mini-GPT (10) (sonder une propriété linguistique).
7. **Dossier de conformité** : choisir un scénario fictif à haut risque (par exemple votre modèle tabulaire utilisé pour un crédit), produire la documentation technique AI Act (annexe IV), le système de gestion des risques, la gouvernance des données, le plan de contrôle humain et de journalisation, l'évaluation d'impact RGPD ; fiches de modèles et de données mises au niveau pour tous vos projets.
8. **Politique de la plateforme** : politique d'usage responsable de AI Lab Hub (ce que vos démos font, ne font pas, données conservées), avertissements dans les démos génératives, procédure de signalement.

Compétences acquises

- Sécurité offensive et défensive des systèmes d'IA classiques et des LLM, avec métriques.
- Audit d'équité et compréhension des arbitrages mathématiques et éthiques.
- Explicabilité appliquée et évaluée, pas décorative.
- Maîtrise du cadre réglementaire européen et production de documentation de conformité.
- Pont entre IA et sécurité : un profil rare et très demandé (AI security, AI governance).

Documentation et mise en valeur

Livrables : rapport de red teaming (attaques, taux, défenses, résiduel), rapport d'équité, rapport d'explicabilité, dossier de conformité AI Act complet, fiches mises au niveau, politique de la plateforme. Sur la plateforme commune : une section « Audit et confiance » avec un **tableau attaques × défenses × taux résiduel** et les rapports téléchargeables ; chaque fiche de modèle affiche désormais une section robustesse et équité. Un dossier AI Act complet, même sur un scénario fictif, est une pièce de portfolio très différenciante à l'heure où les entreprises cherchent ces compétences.

18 · IA multimodale : vision-langage et compréhension de documents

Domaine	Multimodal · vision et langage
Niveau	Expert
Durée indicative	5 à 6 semaines
Valorisation	Plateforme dédiée possible : service d'analyse de documents (factures, schémas, tableaux) et recherche d'images par texte

Objectif et exigences

Relier vision et langage : entraîner un modèle contrastif type CLIP à petite échelle, construire une recherche d'images par texte, fine-tuner un modèle vision-langage (VLM) pour une tâche de compréhension de documents — extraction structurée depuis des factures, des formulaires, des schémas réseau ou des captures de tableaux de bord — et étendre le RAG (12) au multimodal (recherche dans des PDF avec figures et tableaux). Exigences : CLIP miniature entraîné et évalué (recherche zéro-shot), moteur de recherche d'images fonctionnel, pipeline de compréhension de documents comparant OCR + LLM, modèles spécialisés (LayoutLM/Donut) et VLM génératifs sur un jeu annoté avec métriques d'extraction, RAG multimodal avec évaluation, et analyse des hallucinations visuelles.

Notions à rechercher et apprendre

- **Apprentissage contrastif** : CLIP (deux encodeurs, perte InfoNCE, température, taille de lot), espace d'embedding partagé, classification zéro-shot par prompts, SigLIP ; biais et limites (comptage, relations spatiales, texte dans l'image).
- **Modèles vision-langage génératifs** : architecture (encodeur visuel + projecteur + LLM : LLaVA, Qwen-VL, PaliGemma, Idefics, Florence-2), tokens visuels et résolution, entraînement par étapes (alignement puis instruction), hallucinations visuelles et leur évaluation (POPE), fine-tuning avec LoRA.
- **Compréhension de documents** : OCR classique (Tesseract) et neuronal (PaddleOCR, docTR, TrOCR), analyse de mise en page, modèles multimodaux de documents (LayoutLMv3, Donut sans OCR, Pix2Struct), extraction d'informations clés (KIE), questions-réponses sur documents (DocVQA), extraction de tableaux, sorties structurées (JSON contraint) ; documents en français.
- **Recherche multimodale** : indexation d'images par embeddings CLIP, recherche texte→image et image→image, recherche dans des pages de PDF comme images (ColPali — approche « late interaction » sur des patchs), RAG multimodal (figures et tableaux dans le contexte, VLM comme générateur).
- **Autres modalités** : audio-langage (Whisper + LLM, projet 14), vidéo (échantillonnage temporel), génération texte→image (projet 09) — pour situer l'ensemble.
- **Évaluation** : rappel@k pour la recherche, exactitude par champ et F1 pour l'extraction (ANLS pour DocVQA), taux d'hallucination, robustesse aux documents dégradés (scan, rotation, faible résolution), coût par page.

Outils, plateformes et ressources

- **Bibliothèques** : open_clip (entraînement CLIP), Hugging Face Transformers (CLIP, SigLIP, LLaVA, Qwen2-VL, PaliGemma, LayoutLMv3, Donut, Florence-2),

PEFT, Byaldi/ColPali, docTR ou PaddleOCR, Tesseract, PyMuPDF/pdf2image, Outlines ou le mode JSON du LLM pour les sorties contraintes, FAISS/Qdrant.

- **Données** : Flickr8k/COCO Captions (CLIP miniature), vos propres images annotées ; documents : FUNSD, CORD (reçus), DocVQA, SROIE, RVL-CDIP ; **vos propres documents** : factures et devis anonymisés, schémas réseau de vos catalogues, captures Grafana, tableaux de mesures télécoms — annotés en JSON (Label Studio).
- **Matériel** : GPU Colab pour CLIP miniature et fine-tuning LoRA d'un VLM 2-3B ; VLM 7B en QLoRA sur 16 Go.
- **Ressources** : article CLIP (Radford et al., 2021), LLaVA (Liu et al., 2023), Donut, LayoutLMv3, ColPali (Faysse et al., 2024), cours Hugging Face Community Computer Vision (section multimodale), blog Hugging Face « Vision Language Models Explained », documentation open_clip, benchmark DocVQA et POPE, tutoriels Qwen-VL fine-tuning.

Étapes de réalisation

1. **Théorie** : fiches InfoNCE (implémenter la perte et vérifier), architecture VLM, OCR vs sans OCR, ColPali ; lire CLIP et LLaVA.
2. **CLIP miniature** : entraîner avec open_clip un petit CLIP (ResNet/ViT-small + petit transformer texte) sur Flickr8k ou COCO, évaluer la recherche texte→image (Recall@1/5/10) et la classification zéro-shot sur CIFAR-10 ; comparer au CLIP pré-entraîné ; explorer les échecs (comptage, relations).
3. **Recherche d'images** : indexer vos photos (ou un corpus libre) avec CLIP pré-entraîné dans FAISS, interface de recherche par texte en français (modèle multilingue ou traduction), image→image, mesurer la qualité sur 50 requêtes annotées.
4. **Jeu de documents** : constituer 300+ documents de vos types cibles (factures anonymisées, schémas, captures), annoter les champs à extraire en JSON avec schéma Pydantic, fiche de données, jeu de test avec variantes dégradées (scans, photos de travers).
5. **Pipeline OCR + LLM** : OCR (docTR) → texte avec positions → LLM avec sortie JSON contrainte ; exactitude par champ ; analyse des erreurs d'OCR propagées.
6. **Modèles spécialisés** : fine-tuner Donut (sans OCR) et LayoutLMv3 (avec OCR) sur vos annotations ; comparer exactitude, robustesse aux dégradations, vitesse, coût.
7. **VLM génératif** : évaluer Qwen2-VL / PaliGemma en zéro-shot avec instructions JSON, puis fine-tuner par LoRA sur vos annotations ; mesurer le taux d'hallucination (champs inventés) ; tableau final des quatre approches.
8. **Schémas et tableaux de bord** : cas spécifique : le VLM lit vos schémas réseau (liste des équipements et liens) et vos captures Grafana (valeurs et tendances) — évaluer sur 50 exemples ; c'est ce qui relie ce projet à vos autres catalogues.
9. **RAG multimodal** : indexer les pages de vos PDF (normes, catalogues) avec ColPali, récupérer les pages pertinentes comme images, générer avec un VLM ; comparer avec le RAG texte (12) sur des questions portant sur des figures et tableaux ; évaluation avec le protocole du projet 12 étendu.
10. **Service** : API d'extraction de documents (upload → JSON validé) avec coût par page, démo Gradio, fiche de modèle avec limites (types de documents, langues, dégradations).

Compétences acquises

- Apprentissage contrastif et représentations multimodales.
- Modèles vision-langage : usage, fine-tuning, évaluation des hallucinations.
- Compréhension de documents de bout en bout — un cas d'usage industriel majeur (automatisation administrative).
- Recherche multimodale et RAG sur documents riches.
- Comparaison méthodique d'approches très différentes sur une même tâche annotée.

Documentation et mise en valeur

Livrables : CLIP miniature et son évaluation, moteur de recherche d'images, jeu de documents annoté (publié si anonymisé), tableau comparatif OCR+LLM / Donut / LayoutLM / VLM, RAG multimodal évalué, API. Plateforme dédiée envisageable : un **service d'analyse de documents** où le visiteur dépose une facture ou un schéma et obtient le JSON extrait avec les zones surlignées, plus la recherche d'images par texte sur votre galerie. Le tableau comparatif à quatre approches avec robustesse et coût est le livrable central.

19 · Passage à l'échelle : entraînement distribué et inférence optimisée

Domaine	Performance · systèmes pour l'IA
Niveau	Expert
Durée indicative	5 à 6 semaines
Valorisation	Plateforme commune : rapports de performance, courbes de passage à l'échelle, coûts

Objectif et exigences

Comprendre et optimiser ce qui se passe entre le modèle et le matériel : profiler un entraînement, éliminer les goulets d'étranglement, exploiter la précision mixte et la compilation, entraîner sur plusieurs GPU (données, tenseurs, FSDP), puis optimiser l'inférence (fusion d'opérateurs, quantification, TensorRT, batching continu, cache KV, décodage spéculatif) et mesurer coûts et efficacité énergétique. Exigences : profils avant/après avec accélération mesurée, entraînement multi-GPU du mini-GPT (10) avec courbe d'efficacité de passage à l'échelle, comparaison DDP vs FSDP, un noyau GPU personnalisé écrit en Triton, optimisation d'inférence sur un CNN (05) et un LLM (11) avec tableau latence/débit/coût, et rapport d'efficacité en tokens par euro et par joule.

Notions à rechercher et apprendre

- **Matériel** : architecture d'un GPU (SM, mémoire HBM, cache, tensor cores), bande passante vs calcul, intensité arithmétique et modèle roofline, opérations liées à la mémoire vs au calcul, interconnexions (PCIe, NVLink), CPU/GPU et transferts.
- **Profilage** : PyTorch Profiler, Nsight Systems, traces, temps GPU vs CPU, chargement de données comme goulet (DataLoader, workers, formats, prétraitement GPU), synchronisations implicites.
- **Optimisation d'entraînement** : précision mixte (FP16/BF16, loss scaling), torch.compile et fusion d'opérateurs, attention efficace en mémoire (FlashAttention), gradient checkpointing, taille de lot et accumulation, optimiseurs à mémoire réduite (8 bits), formats de données (WebDataset, Parquet, mmap).
- **Distribution** : parallélisme de données (DDP, all-reduce, ring), ZeRO et FSDP (partitionnement des états), parallélisme de tenseurs, de pipeline, de séquence/contexte, parallélisme d'experts, communications collectives (NCCL), efficacité de passage à l'échelle (forte/faible), tolérance aux pannes et points de reprise, lois d'échelle et budget de calcul (lien projet 10).
- **Noyaux** : CUDA en notion, Triton (écrire un noyau fusionné en Python), pourquoi la fusion aide (moins d'allers-retours mémoire), comment FlashAttention fonctionne.
- **Inférence** : export et graphes (ONNX, TorchScript, torch.export), TensorRT, fusion et quantification (INT8 avec calibration, FP8), distillation et élagage, pour les LLM : batching continu, PagedAttention, cache KV et sa mémoire, décodage spéculatif, quantification (projet 11), serveurs (vLLM, TensorRT-LLM, SGLang, TGI), métriques (TTFT, tokens/s par requête et agrégés, latence p99).
- **Coûts et énergie** : coût horaire GPU cloud vs local, utilisation réelle, coût par million de tokens et par 1 000 images, mesure de la consommation (nvidia-smi, CodeCarbon), efficacité énergétique, empreinte carbone de l'IA (ordres de grandeur).

Outils, plateformes et ressources

- **Outils** : PyTorch (Profiler, DDP, FSDP, compile, AMP), Triton, Nsight Systems, Accelerate et DeepSpeed (ZeRO), FlashAttention, ONNX Runtime, TensorRT et TensorRT-LLM, vLLM, SGLang, k6 ou locust pour la charge, CodeCarbon, nvidia-smi/DCGM exporter vers Prometheus.
- **Matériel** : location de nœuds multi-GPU à l'heure (RunPod, Lambda, Vast.ai : 2 à 8 GPU pour quelques euros de l'heure — budgetiser 30 à 60 € pour le projet) ; le reste sur un seul GPU ou Colab.
- **Ressources** : « Making Deep Learning Go Brrrr From First Principles » (Horace He — lecture fondatrice sur le roofline), « The Ultra-Scale Playbook » (Hugging Face — entraînement à grande échelle, gratuit et complet), article ZeRO (Rajbhandari et al.), FlashAttention (Dao et al.), Megatron-LM, tutoriels Triton officiels, documentation PyTorch FSDP et Profiler, article PagedAttention/vLLM (Kwon et al., 2023), décodage spéculatif (Leviathan et al.), cours Stanford CS336 (sections systèmes), « GPU MODE » (communauté et cours sur YouTube), *Programming Massively Parallel Processors* (Hwu, Kirk) pour la culture CUDA.

Étapes de réalisation

1. **Théorie** : fiches roofline (calculer l'intensité arithmétique d'une multiplication de matrices et d'une attention et prédire si elles sont liées à la mémoire ou au calcul), collectives, ZeRO, cache KV (calculer sa mémoire pour un 7B à 8 k tokens) ; lire Horace He et l'Ultra-Scale Playbook.
2. **Profilage d'un entraînement** : profiler l'entraînement du mini-GPT (10) et du CNN (05) : identifier le goulet (souvent le chargement de données ou les petites opérations), trace annotée ; corriger étape par étape (workers, pin memory, prétraitement GPU, taille de lot) ; mesurer l'accélération à chaque étape (tableau).
3. **Optimisations sur un GPU** : BF16, torch.compile, FlashAttention, gradient checkpointing, optimiseur 8 bits ; mesurer débit (tokens/s ou images/s) et mémoire pour chaque combinaison ; atteindre un pourcentage mesuré du pic théorique du GPU (MFU).
4. **Noyau Triton** : écrire un noyau fusionné (par exemple softmax ou LayerNorm fusionné, puis une attention simplifiée), le vérifier contre PyTorch, le comparer en vitesse ; expliquer pourquoi il est plus rapide (ou pas).
5. **Multi-GPU** : louer un nœud 2 puis 4 puis 8 GPU ; entraîner le mini-GPT avec DDP puis FSDP ; mesurer le débit et tracer l'efficacité de passage à l'échelle ; comparer mémoire par GPU DDP vs FSDP ; tester la reprise après une panne simulée ; documenter les coûts réels de la session.
6. **Inférence CNN** : exporter le détecteur (05) en ONNX puis TensorRT (FP16, INT8 avec calibration), mesurer latence et débit par taille de lot sur GPU et CPU, comparer à PyTorch nu ; tableau précision/latence/débit.
7. **Inférence LLM** : servir le modèle (11) avec Transformers nu, vLLM, SGLang et TensorRT-LLM ; mesurer TTFT, tokens/s et p99 sous charge croissante ; effet du batching continu, de la quantification, du décodage spéculatif (petit modèle brouillon) ; tableau et courbes débit-latence.
8. **Coûts et énergie** : mesurer la consommation pendant entraînement et inférence, calculer tokens par euro et par joule pour chaque configuration, comparer local vs cloud vs API propriétaire ; rapport d'efficacité.

Compétences acquises

- Compréhension du matériel GPU et raisonnement par le modèle roofline.
- Profilage et optimisation d'entraînements, écriture de noyaux GPU en Triton.
- Entraînement distribué (DDP, FSDP) avec mesures d'efficacité et de coût.
- Optimisation d'inférence pour la vision et les LLM, serveurs d'inférence modernes.
- Culture FinOps et énergétique de l'IA — le profil « ML systems / performance », rare et très bien valorisé.

Documentation et mise en valeur

Livrables : traces de profilage annotées avant/après, tableau d'accélération par optimisation, noyau Triton publié avec benchmark, courbe d'efficacité de passage à l'échelle, tableaux d'inférence, rapport coûts/énergie. Sur la plateforme commune : une section « Performance » avec les courbes et un **calculateur de coût d'inférence** alimenté par vos mesures. Un article « j'ai entraîné mon GPT sur 8 GPU : efficacité, pièges et facture » est très apprécié ; le noyau Triton et le MFU atteint sont des preuves techniques fortes.

20 · Produit IA complet : de la donnée à la gouvernance

Domaine	Synthèse : produit, ingénierie, exploitation, gouvernance
Niveau	Expert (projet de synthèse)
Durée indicative	8 à 10 semaines
Valorisation	Plateforme dédiée : le produit lui-même, ouvert à de vrais utilisateurs

Objectif et exigences

Concevoir, construire, déployer et gouverner un produit d'IA complet destiné à de vrais utilisateurs, en réutilisant les briques des dix-neuf projets : cadrage avec les utilisateurs, choix d'approche justifié par des expériences, modèles (classiques et LLM) combinés, application, plateforme MLOps, monitoring, sécurité, conformité, mesure de la valeur réelle et amélioration continue. Suggestion cohérente avec vos trois portfolios : un **copilote d'exploitation réseau et télécom** qui ingère métriques, journaux et tickets de votre laboratoire (ou d'une communauté), détecte les anomalies (04), prévoit les charges (07), corrèle et classe les incidents (06), répond aux questions sur la documentation (12), propose des diagnostics avec validation humaine (13), lit les schémas et captures (18), le tout servi et surveillé par la plateforme MLOps (16), audité (17) et optimisé (19). Toute autre idée à laquelle vous tenez convient, à condition d'avoir des utilisateurs réels. Exigences : au moins cinq utilisateurs réels pendant quatre semaines, métriques de valeur définies avant le lancement et mesurées, au moins trois modèles en production, évaluation continue, dossier de gouvernance complet, post-mortem et feuille de route.

Notions à rechercher et apprendre

- **Produit IA** : découverte utilisateur, formulation du problème (faut-il de l'IA ? quelle métrique métier ?), définition des métriques de succès (métrique métier, métrique modèle, garde-fous), estimation de la valeur, choix entre règles / ML classique / LLM (« la plus simple qui marche »), gestion des attentes, design d'interaction avec l'incertitude (afficher la confiance, permettre la correction), boucle de retour utilisateur.
- **Architecture de système IA** : composition de modèles, orchestration, données en flux et par lots, cohérence entraînement/service, latence de bout en bout, dégradation gracieuse (que se passe-t-il si un modèle tombe ?), coûts.
- **Évaluation continue** : jeux d'évaluation vivants alimentés par la production, étiquetage par les utilisateurs, régression, tests A/B et mesures de valeur, distinction corrélation/causalité dans l'évaluation d'impact.
- **Gouvernance** : registre des modèles et des risques, revue avant mise en production, documentation (fiches, dossier AI Act), traçabilité des décisions, gestion des incidents IA, rôles et responsabilités, RGPD.
- **Organisation et communication** : présenter un système d'IA à des décideurs (valeur, risques, limites), rédiger une étude de cas, gérer la dette technique et la feuille de route.

Outils, plateformes et ressources

- **Outils** : tous ceux des projets précédents ; en plus : outil de retours utilisateurs intégré, tableau de bord de valeur (Grafana ou Metabase), gestion de projet (GitHub Projects), registre de risques.

- **Utilisateurs** : votre entreprise ou école, une association, une communauté de laboratoires, des pairs qui ont suivi les catalogues réseaux/télécoms — cinq personnes qui utilisent réellement l'outil valent plus que mille visiteurs.
- **Ressources** : *Designing Machine Learning Systems* (Huyen — intégralement), *Machine Learning Design Patterns* (Lakshmanan et al.), *Building Machine Learning Powered Applications* (Ameisen — le cycle produit), *AI Engineering* (Huyen, 2025 — applications LLM), « Rules of Machine Learning » (Martin Zinkevich, Google — 43 règles pragmatiques), « What is ML Ops » et études de cas d'équipes ML (blogs d'ingénierie Netflix, Uber, Spotify, Airbnb), guides de gouvernance CNIL et Commission européenne, People + AI Guidebook (Google) pour le design.

Étapes de réalisation

1. **Découverte (semaine 1)** : entretiens avec 5 à 10 utilisateurs potentiels, problèmes réels observés, choix du périmètre minimal, métriques de valeur (par exemple : temps moyen de diagnostic d'un incident, taux d'incidents détectés avant plainte, satisfaction), métriques de garde-fou (faux positifs, hallucinations, coût) ; document de cadrage.
2. **Décision d'approche (semaine 2)** : pour chaque fonctionnalité, comparer règles / ML classique / LLM par une expérience rapide sur données réelles ; retenir la plus simple qui atteint l'objectif ; architecture cible avec dégradation gracieuse ; évaluation des risques initiale (AI Act : classification du système).
3. **Données et évaluation (semaine 3)** : pipelines d'ingestion (métriques, journaux, tickets, documents), fiches de données, jeux d'évaluation par fonctionnalité, protocole d'évaluation continue, consentement et minimisation des données personnelles.
4. **Construction (semaines 4 à 6)** : intégrer et adapter les modèles (04, 06, 07, 12, 13, 18), application (Next.js) avec affichage de l'incertitude, des sources et des boutons de correction, API, déploiement sur la plateforme MLOps (16) avec monitoring, garde-fous (17), optimisation des coûts (19) ; revue de sécurité et de conformité avant lancement.
5. **Lancement et exploitation (semaines 7 à 10)** : mise en service auprès des utilisateurs, mesures hebdomadaires de valeur et de garde-fous, retours utilisateurs collectés et traités, réentraînements, au moins un test A/B (par exemple deux formulations de diagnostic), gestion d'au moins un incident réel (dérive, panne, réponse erronée) avec post-mortem.
6. **Gouvernance** : dossier complet (registre des modèles, fiches, risques, dossier AI Act si haut risque, RGPD, procédure de revue, journalisation), audit interne selon la grille du projet 17.
7. **Bilan** : rapport de valeur (avant/après sur les métriques définies, avec honnêteté sur ce qui n'est pas prouvé), post-mortem global, coûts totaux (calcul, temps), feuille de route, étude de cas rédigée, présentation de 20 minutes filmée, retex de l'ensemble du catalogue.

Compétences acquises

- Conduite d'un produit d'IA de bout en bout : découverte, choix d'approche, construction, lancement, exploitation, amélioration.

- Composition de plusieurs modèles dans un système fiable avec dégradation gracieuse.
- Mesure de la valeur réelle et évaluation continue en production.
- Gouvernance, conformité et gestion d'incidents IA.
- Communication avec des utilisateurs et des décideurs — la compétence qui transforme un technicien en responsable de produit ou d'équipe IA.

Documentation et mise en valeur

Livrables : document de cadrage, étude comparative des approches, architecture, dossier de gouvernance, tableau de bord de valeur, post-mortems, rapport final, étude de cas, vidéo de présentation. Plateforme dédiée : le **produit lui-même**, avec une page publique « comment il fonctionne, ce qu'il mesure, ses limites » et des témoignages d'utilisateurs. Ce projet est la pièce maîtresse du portfolio IA et le point de convergence des quatre catalogues : il montre que vous savez transformer des compétences en valeur mesurée, en toute transparence.

Synthèse : compétences, plateforme et séquencement

Couverture des compétences

Correspondance entre familles de compétences et projets

Famille de compétences	Projets principaux	Preuves attendues
Mathématiques et fondements	02, 09, 10, 15	Implémentations vérifiées, dérivations, vérification de gradient
Données, évaluation, méthodologie	01, 03, 04, 07, tous	Protocoles écrits, baselines, tableaux d'expériences, intervalles de confiance
ML classique	03, 04, 07	Classement Kaggle, détecteur en production, backtesting
Deep learning par domaine	05, 06, 07, 08, 09, 14, 18	Modèles publiés, comparatifs, démos interactives
LLM et IA générative	09, 10, 11, 12, 13, 18	GPT de zéro, modèle fine-tuné publié, RAG évalué, agent avec harnais
RL	15	Courbes multi-graines, environnement publié, agent vs heuristiques
Industrialisation et performance	05, 11, 16, 19	Plateforme MLOps, incident de dérive rejoué, noyau Triton, courbes de passage à l'échelle
Sécurité, éthique, conformité	13, 17, 20	Rapports de red teaming et d'équité, dossier AI Act
Produit et communication	08, 12, 20, plateforme	Utilisateurs réels, mesures de valeur, articles, étude de cas, vidéos

Plateforme commune ou dédiée : récapitulatif

- **Plateforme dédiée pertinente** : 02 (visualiseur d'apprentissage), 05 (démo caméra), 08 (site de recommandation), 09 (galerie générative), 12 (assistant RAG sur le portfolio), 13 (agent d'exploitation), 14 (assistant vocal), 16 (plateforme MLOps), 18 (analyse de documents), 20 (le produit).
- **Plateforme commune AI Lab Hub** : tous les autres, et la page de tous les projets ; les démos Hugging Face Spaces y sont intégrées, MLflow alimente le tableau d'expériences, la plateforme MLOps (16) alimente le tableau de bord de production, l'assistant RAG (12) sert de moteur de recherche du portfolio.
- **Convergence des catalogues** : les projets 04, 07, 13, 15, 18 et 20 exploitent les données et infrastructures des catalogues réseaux/systèmes/sécurité et télécoms ; l'assistant RAG (12) indexe les quatre catalogues. Un portfolio unique à quatre volets, interconnecté, raconte une histoire cohérente : infrastructure → mesures → intelligence → produit.

Séquencement indicatif sur 12 mois

Séquencement indicatif à raison de 15 à 20 heures par semaine

Période	Projets	Jalon de portfolio
Mois 1-2	01, 02, 03	Laboratoire et MLflow en place, bibliothèque ML from scratch publiée, première compétition Kaggle, AI Lab Hub v1
Mois 3-4	04, 05, 06	Détecteur d'anomalies en direct sur le lab, démo caméra, modèle NLP français publié sur Hugging Face
Mois 5-6	07, 08, 09, 10	Prévisions publiées quotidiennement, site de recommandation, galerie générative, GPT de zéro et son article
Mois 7-8	11, 12, 13, 14	LLM fine-tuné publié, assistant RAG sur le portfolio, agent d'exploitation, assistant vocal
Mois 9-10	15, 16, 17	Agent RL vs heuristiques, plateforme MLOps avec incident de dérive rejoué, rapports d'audit et dossier AI Act
Mois 11-12	18, 19, 20	Service d'analyse de documents, rapports de performance multi-GPU, produit IA avec utilisateurs réels, bilan des quatre catalogues

Trois principes pour que ce portfolio soit crédible. Premièrement, chaque page affiche une baseline et un tableau d'expériences honnête, échecs compris ; c'est ce que regardent les praticiens. Deuxièmement, chaque modèle est essayable en un clic (démo intégrée) et publié avec sa fiche ; un visiteur qui teste retient. Troisièmement, au moins deux modèles tournent réellement en production avec un tableau de bord public sur trente jours : la différence entre « j'ai entraîné un modèle » et « j'exploite des modèles » est celle qui compte le plus pour un employeur. Budget de calcul total estimé pour les 20 projets : entre 100 et 250 € de GPU à la demande si vous n'avez pas de GPU personnel — le reste tient sur Colab, Kaggle et Hugging Face Spaces gratuitement.

Document de conception pédagogique. Durées et coûts sont indicatifs (calcul GPU : 100 à 250 € au total sans GPU personnel). Les modèles et bibliothèques cités évoluent vite : vérifier versions et licences au moment de la réalisation. Tout traitement de données personnelles doit respecter le RGPD ; les modèles publiés doivent être accompagnés d'une fiche de modèle.